

Accenture AG

Equity Insight

Individual practical work of application development apprentice
Sheung Tsung Lam

Part 1: Environment and process	4
1. Preface	4
2. Project environment and scope	4
3. Project organisational structure	5
4. Involved persons and roles	5
5. Initial position	6
6. Functional requirements	6
7. Technical requirements	7
1. Technology stack	7
8. Excluded requirements	7
9. Means and methods	8
1. Project method	8
2. Work results and version control	8
10. Prior knowledge	9
11. Preliminary work	9
12. New learning content	10
13. Time plan	12
14. Work journal	13
1. Tuesday, 07.04.2026	13
2. Wednesday, 08.04.2026	15
3. Thursday, 09.04.2026	16
4. Friday, 10.04.2026	17
5. Tuesday, 14.04.2026	19
6. Wednesday, 15.04.2026	20
7. Thursday, 16.04.2026	21
8. Friday, 17.04.2026	23
9. Tuesday, 21.04.2026	24
10. Wednesday, 22.04.2026	25
15. Personal conclusion	26
Part 2: Project	27
1. Short summary	28

1. Initial situation	28
2. Objective.....	28
3. Implementation	28
4. Result	28
2. Project description.....	29
3. IPERKA	29
1. Inform (Informieren)	30
1. Expected outcome	31
2. Emerged uncertainties.....	31
2. Plan (Planen).....	31
1. Data concept.....	31
2. Frontend structure	33
3. Backend structure.....	35
4. User interface mock-ups	37
5. Backend API endpoint definition:	46
6. Implementation concept.....	47
7. Test concept	48
3. Decide (Entscheiden).....	50
1. Multi-user or single-user.....	50
2. Which Graph library	50
3. Which stock data API	51
4. Vertical or horizontal scrolling	52
5. Backend structure.....	53
4. Realise (Realisieren).....	55
1. Environment and structure.....	55
2. Database	57
3. Backend.....	59
4. Frontend	66
5. Containerisation	68
6. AI usage	69
5. Control (Kontrollieren).....	69

1. Testing.....	69
6. Evaluate (Auswerten)	73
4. List of illustrations	75
5. List of sources	77
6. Glossary.....	77
7. Attachment	78

Part 1: Environment and process

1. Preface

This document contains the report for my individual practical work (IPA) in the field of application development. The project was carried out from 07.04.2026 to 22.04.2026 and documents the planning, implementation, and evaluation of the application Equity Insight.

The aim of this report is to present both the project process and the resulting technical solution in a structured and traceable way. For this reason, the report is divided into two parts. The first part describes the environment, organisation, time planning, and work process of the IPA. The second part documents the project itself according to the IPERKA methodology, including the analysis, planning, decisions, implementation, and evaluation.

This report is intended for the expert team, the supervisor, and other technical readers who need a clear understanding of the project, its implementation, and the reasoning behind the most important decisions.

2. Project environment and scope

The project was carried out in the context of my apprenticeship in application development and was implemented as an individual practical work. The task was to design and implement a local, single-user stock tracking application called “Equity Insight” within the official IPA time frame.

The project environment was defined by both technical and organisational constraints. From a technical perspective, the project had to be implemented as a full-stack application using Angular and TypeScript in the frontend, Java and Spring Boot in the backend, and a PostgreSQL database as the database layer. Apache ECharts was selected for chart visualisation, and Podman was used for containerisation of the database. Git was used for version control, and the selected IDE was IntelliJ IDEA from JetBrains.

From an organisational perspective, the project was carried out independently by the candidate (me), with support from the supervisor and feedback from the expert team. The project also had to follow the formal structure and requirements of the IPA.

The functional scope of the project included watchlist management, stock handling, stock price visualisation, comment management, and theme switching between light and dark mode. At the same time, several features were intentionally excluded from the scope, including authentication, multi-user support, notification, financial transactions, and mobile device support.

3. Project organisational structure

The project organisational structure for this IPA is intentionally simple. The project is to be carried out by the candidate (me) as individual practical work, supported by a supervisor and reviewed by both the supervisor and the expert team.

The candidate (me) will be responsible for the analysis, planning, decision making, implementation, documentation, and preparation for final delivery. The supervisor acts as a professional contact person in the company environment and supports the project through feedback, prioritisation, approval of important organisational changes, and general advisory. The expert team is responsible for reviewing the work in the context of the requirements as well as providing guidance regarding formality and expectations.

4. Involved persons and roles

Candidate – Sheung Tsung (Masa) Lam

The candidate is responsible for carrying out the IPA independently. This includes risk analysis, planning, decision making, implementation, testing, and documentation.

Supervisor – Stefan Copetchi

The supervisor supports the project from the company's side. The work includes clarifying expectations, reviewing progress, providing technical advice, approving organisational adjustments, and at the end assessing the project.

Expert team – Gajanthan Vasantharuban and Reto Hablützel

The expert team reviews the IPA, and provides feedback and tips during meetings.

5. Initial position

This project begins with no existing application, logic or anything more sophisticated than surface level conceptualisation.

The goal application is a local application intended for the single local user. Because the stock market data intended to be used in this application is publicly available and the application is not designed to be cloud-based. Authentication or authorisation functionalities are intentionally excluded.

One very critical condition is for the implementation to be legally compliant due to the sensitivity of stock data. Through research, I found out that free and legitimate stock data sources only provide delayed stock data. For this reason, real-time stock data tracking is out of the scope of this project, though given a larger budget and less limited time, this would have been possible.

6. Functional requirements

- Watchlist management: The application shall allow the users to create, view, update and delete watchlists. Each watchlist shall have a title and contain any number of stocks. The same stock may be added to multiple watchlists, but the same stock shall not be added more than once to the same watchlist.
- Stock search: The application shall allow the user to search for stocks by their ticker symbols or company names.

- Historical stock price visualisation: The application shall allow users to choose from a defined time range, and display historical price data within that time range.
- The application shall allow users to create, view, update and delete comments for an individual stock. Each comment shall contain a time stamp, indicating when it was created or last updated.
- Theme toggle: The application shall provide an option for its users to choose from dark mode and light mode at run time.
- Persistent data: The application shall persist locally managed user data.

7. Technical requirements

1. Technology stack

- Frontend: Angular and Typescript
- Backend: Java and Spring Boot
- Database: PostgreSQL
- Chart visualisation: Apache ECharts
- Containerisation: Podman
- Stock data API: Stooq

8. Excluded requirements

- Real-time stock data tracking: Due to legal restraints, free and legitimate stock data sources only provide delayed data.
- Stock price notifications: This app feature was decided to be excluded, mainly because of my lack of experience with related technologies.
- Authentication and multi-user functionality: This application is designed to only be run locally on each machine, rendering the implementation of these functionalities more time consuming than necessary.
- Financial transactions: This functionality requires agreements with banks and stock markets, which is unfortunately not realistic without the support of a large budget
- Mobile device support: This will not be implemented for being too time consuming

9. Means and methods

1. Project method

This project will be carried out according to the IPERKA method. It was chosen because it provides a clear and structured sequence from analysis to evaluation, which is suitable for an IPA with defined requirements, limited time and strong documentation focus.

The following tools supported the project work:

- IntelliJ IDEA for backend and frontend development
- Angular CLI for frontend deployment
- Spring Initializr for backend project setup
- PostgreSQL as database
- Podman for containerisation
- Figma for mock-ups and diagrams
- Draw.io for diagrams
- Postman and Swagger for endpoint testing
- ChatGPT for support in examples, explanations and technical orientation

2. Work results and version control

Git was used as the version control system for this project. It was used to document changes in the code base continuously and to keep development process traceable.

A GitHub repository was created for the project in order to back up the work results and keep the project history available on a server. During development phase, the changes were saved regularly in commits. This made it possible to separate important steps such as code base setup, entity modelling, CRUD functionality implementation, UI implementation etc. and document changes more clearly.

10. Prior knowledge

Before this IPA, I already had practical experience in full-stack software development from my apprenticeship and internal company tasks. I had worked on both frontend and backend topics.

In the frontend, I knew the basics of Angular and TypeScript. I had experience with components, services, and the general structure of a frontend application. In the backend, I had more advanced experience with Java and Spring Boot. This included working with REST endpoints, service and repository layers, and simple DTOs. I also had basic knowledge of relational databases and SQL.

I was also familiar with Git for version control and with general software development principles such as modular structure, separation of concerns, and implementing requirements in a structured way. This knowledge gave me a good foundation for the IPA.

At the same time, there were some areas where I had less experience. These included containerisation with Podman, using the IPERKA methodology in a complete project, and realising user interfaces based on mock-ups.

The prior skills make this project feasible, while the topics where I am less familiar with became part of the new learning content during the IPA.

11. Preliminary work

To ensure this project was viable, some preliminary work had to be done.

One example of such is the research for a fitting stock data source. The planned application required historical stock data, but the data source also had to be free and legally usable.

Another part of this was the research for a suitable chart library for displaying stock prices more clearly. Since stock visualization belongs to the core functionalities of the application, I had to identify a library that could support interactive line charts.

In addition, a minimal, surface level UI mock-up was also created as a design basis. In the planning phase, this will be improved upon with mock-ups that are more detailed and are closer to the realistic implementation.

Overall, the preliminary work done helped reduce uncertainties before the start of this project, and set a solid foundation for the project execution.

12. New learning content

During this IPA, I gained new knowledge in several technical and methodological areas.

One important learning area was the structured execution of a software project according to the IPERKA methodology. In previous work, I had already developed software components, but during this IPA I had to carry a project from the initial analysis and planning phase through decision making, implementation, documentation, and preparation for testing in a much more structured and traceable way.

A second major learning area was documentation. I learnt how important it is to document not only the technical result, but also the reasoning behind design decisions, the comparison of variants, the tracking of planned versus actual work, and the handling of problems and risks. This became especially relevant after feedback from the expert, which showed that IPA documentation must be much more explicit and criteria-oriented than normal project notes.

From a technical point of view, I also deepened my understanding of backend architecture with Spring Boot. In particular, I improved my knowledge of the implementation of entity relationships in Spring Boot. On the frontend side, I gained further practical experience with Angular, especially in the area of component interaction, services, state handling, and user interface implementation.

Another important learning experience was the handling of technical risk and late scope adaptation. When the originally planned external stock data source could no longer be used as intended, I had to adapt the solution and switch to backend-generated mock data. This taught me the importance of prioritising a stable and testable solution over an originally planned but risky feature.

13. Time plan

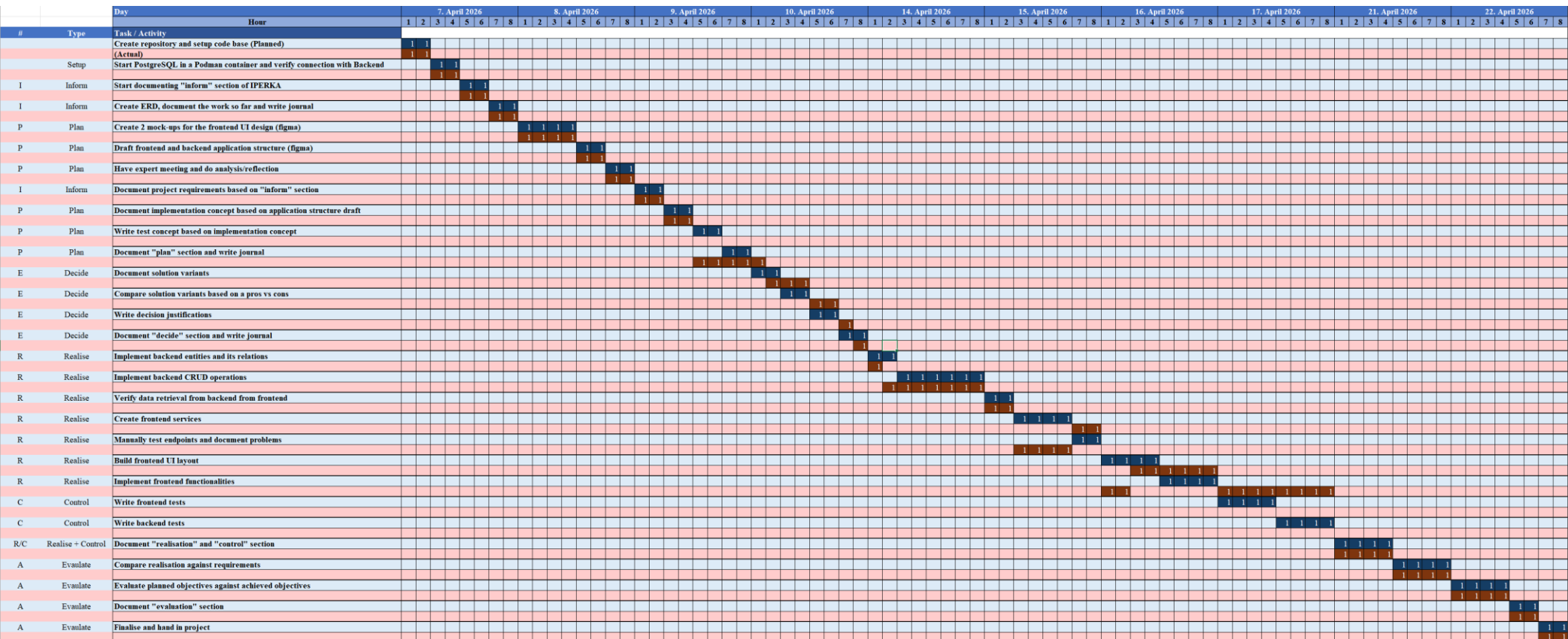


Figure 1 - time plan

This IPA was planned over ten workdays using an hour raster. The structure of the plan follows the IPERKA methodology in the order of inform, plan, decide, realise, control and evaluate. The plan was updated continuously during the IPA and includes a comparison between planned and actual work.

Legend:

- Navy = planned time
- Crimson = actual time

14. Work journal

This section documents the daily progress of the IPA. It includes the planned work, the work actually carried out, a short description of the work, achieved and missed goals, problems that occurred, and deviations from the original time plan. The purpose of this journal is to make the project trackable and show how the decisions, risks and changes were handled.

1. Tuesday, 07.04.2026

Planned work for today was to:

- Create repository and setup code base (2h)
- Start PostgreSQL in a Podman container and verify connection with backend (2h)
- Start documenting “inform” section of IPERKA (2h)
- Create ERD (1h)
- Write journal (1h)

I first created the GitHub repository for the project. I also managed to initialise the backend with Spring Initializr and the frontend with Angular CLI. Then I also started the PostgreSQL database with a Podman command in a Podman container.

A connection test between the backend and the database initially failed. The first solution I tried was just to reinitialise both the backend and the container. As it turned out, it was just what I needed to do.

I then also began writing the “Inform” section of my documentation based on the project description. I found a potential risk, which was about the “search by company name” functionality, because Stooq only accepts ticker symbols in requests.

Additionally, I created an ERD for my data structure, and added the required modules to the backend application.

Achieved goals:

Repository created; Frontend application initialised; Backend application initialised; PostgreSQL database container initialised; Backend to database connection established and tested; ERD for data structure created

Missed goals:

“Inform” section not quite finished, UI design mock-ups not yet created.

Problems:

Backend to database connection failed initially, but the issue was resolved after reinitialising the backend and the container

Support used:

Spring Initializr; Angular CLI; Podman; Project description; Own prior knowledge

Comparison with time plan:

Most of the planned setup work was completed as intended. However, the connection issue reduced the time available for documentation, which is why the “Inform” section could only be started and not fully completed.

2. Wednesday, 08.04.2026

Planned work for today was to:

- Create 2 mock-ups for the frontend UI design (4h)
- Draft frontend application structure (2h)
- Have expert meeting and do analysis and reflection (2h)

Today I created two variants of frontend UI mock ups using Figma. I mainly wanted to visualise the concept in my head to better compare them and see whether they are viable. The first variant was based on my original design idea, which features mainly vertical scrolling elements. The second variant uses more horizontal scrolling. Purely based on appearance, I currently prefer the second variant. I however intentionally postponed the final decision making for design choices, because architectural implications should also be considered, which is currently lacking. At this stage though, the first variant appears to be more feasible.

After that, I drafted the frontend and backend structure based on the previously created ERD as well as the technical requirements. During this work, I refined the initial concepts further and made two structural variants for the frontend and three for the backend, with each having its own advantages and disadvantages. The final decision making will be documented in the “decide” section of this document.

Near the end of the day, I had a scheduled meeting with the main expert and my supervisor. During this, the main expert provided me with tips he thought I might need based on his experience of grading IPAs. I also realised that my daily planning so far was not sophisticated enough according to IPA standards. So I spent the rest of this workday creating a new, more detailed and more accurate version according to the standards, which was then approved by my supervisor after discussing with him.

Achieved goals:

two UI design variants created in Figma; frontend structure drafted in two variants; backend structure drafted in three variants; feedback received from the main expert; daily planning revised and approved by my supervisor after discussion

Problems:

The original daily planning had to be redone to better match IPA criteria

Support used:

Figma; ERD from previous day; feedback from main expert; feedback from supervisor

Comparison with time plan:

The planned design and structure work was completed in substance. However, part of the workday had to be used for improving the planning after expert feedback, which was necessary to better align the project with IPA requirements.

3. Thursday, 09.04.2026

Planned work for today was to:

- Document project requirements based on “inform” section (2h)
- Document implementation concept based on application structure draft (2h)
- Write test concepts based on implementation concept (2h)
- Document “plan” section (2h)

Today I mainly continued the planning phase. I refined the functional requirements, technical requirements and excluded requirements to further clarify the project scope, which would be beneficial for the planning phase.

After that, I further developed the data concept. I then documented the evolution from the initial data model idea to a revised version with a dedicated comment entity, and finally to an improved version using a junction table between the watchlist table and the stock table. The changes were made in order to avoid data duplication.

Furthermore, I expanded the application structure concepts for both the frontend and the backend. For the frontend, I documented how the initial single-page approach, that later evolved into a more modular structure to better separate responsibilities. For the backend I documented multiple variants, each with its own advantages and disadvantages. In the decision phase, the structure to actually follow when implementing will be chosen.

I then drafted the test concept based on the implementation concept. This included thinking about which areas of the application with need to be tested, including which areas of the application will need to be tested and which test types are suitable.

Achieved goals:

Functional requirements refined; Technical requirements refined; Excluded requirements refined; Data concept developed further; Frontend structure concept expanded; Backend structure variants documented further; Test concept drafted; “Plan” section expanded

Problems:

No major technical problem occurred, but the planning and documentation work required more detail than initially expected

Support used:

previous project description; ERD; own prior knowledge; supervisor input from earlier discussion

Comparison with time plan:

The workday remained within the planned scope of the planning phase. The planned documentation and concept work was completed in substance, although some parts required more detail than originally expected.

4. Friday, 10.04.2026

Planned work for today was to:

- Document solution variants (2h)
- Compare solution variants based on advantages and disadvantages (2h)
- Write decision justifications (2h)
- Document decide section (2h)

Today I mainly continued the transition from the planning phase into the decision phase. I further refined the backend concept and documented an additional backend structure variant. While defining the planned API endpoints, I realised that there is also a possible alternative data handling approach in which stock related fields would not necessarily need to be stored as independent stock entities. Instead, its fields could be distributed between the watchlist entity and the comment entity. I documented this as an additional backend implementation option so it can be evaluated properly before implementation starts.

After documenting the variants, I compared them using their advantages and disadvantages. The purpose is to reduce uncertainty and prepare for the decision making. I then also began writing decision justifications for them.

I also drafted the planned backend API endpoints for the watchlist, stock and comment controllers. This helped me verify whether backend application structure draft is consistent with the requirements and whether the separation of responsibilities make sense.

In addition, I continued preparing for the decision phase. I documented the decision context for the stock data API and the user interface variant. This was necessary so that my decisions can be traceable for documentation.

Achieved goals:

backend structure concept refined further; additional backend variant documented; planned backend API endpoints drafted; started writing decision justifications; started documenting “decide” section

Problems:

Today was primarily used for conceptualisation and decision making, rather than implementation. However, the work today improves traceability and reduces the risk of starting implementation on an unstable basis.

Support used:

Backend structure drafts; Requirements documentation; Own analysis; Supervisor and Expert feedback from earlier meetings

Comparison with time plan:

The workday matched the planned shift from the planning phase into the decision phase. The intended conceptual and decision-related tasks were completed in substance, while the actual implementation was intentionally postponed until the structure and choices were documented more clearly.

5. Tuesday, 14.04.2026

Planned work for today was to:

- Implement backend entities and its relations (2h)
- Implement backend CRUD operations (6h)

Today I started the realising phase. I began implementing the backend application, mainly focusing on the three main entities and the adequate CRUD operations for them. I firstly reviewed the data concepts created in the previous phase before implementing.

I originally planned two hours for the implementation of the entities and their relationships. I did turn out to use around the same amount of time, but scattered rather than in a single sitting.

As for the implementation of the CRUD operations for those entities, I planned six hours, and also actually used six hours. It was during the implementation of the CRUD operations where I noticed flaws in my implementation for the entity relations, which caused me to spend time on that again.

Overall, today marked the day where I truly started implementing features.

Achieved goals:

Backend entities and relationships implemented; Most relevant CRUD operations implemented;
Most important bugs corrected

Problems:

While implementing CRUD operations, flaws in the entity relations became visible and had to be corrected during the same workday

Support used:

Previous data concept; Backend structure draft; Own prior knowledge; Spring Boot documentation where required

Comparison with time plan:

The workday followed the planned start of backend implementation. Both main tasks were completed in substance, although some of the time planned for CRUD implementation also had to be used for correcting entity relation issues discovered during coding.

6. Wednesday, 15.04.2026

Planned work for today was to:

- Verify data retrieval from backend (2h)
- Implement frontend services (4h)
- Test backend endpoints with postman (2h)

Today I started the work with verifying the backend endpoints I implemented the day prior using a browser. I managed to complete that within the two hours I planned

Then I started testing the endpoints with mock data with Postman. I originally planned only two hours for this task, but it ended up taking four hours because I had problems aligning the data sent to the backend and the data my backend would expect. I managed to solve this issue via trial and error at the end.

After testing was done, I started implementing frontend services. Thanks to the successful testing of the backend endpoints with Postman, this task wasn't too hard. I managed to complete it within the two hours left of today.

Achieved goals:

Endpoints verified; Endpoints tested with mock data; Frontend services created

Problems:

Manual testing took longer than expected due to mismatch between the sent data and the data expected by the backend. However, solving this issue made the following frontend service implementation easier.

Support used:

Browser endpoint checks; Postman; Backend implementation from previous day; Own prior knowledge

Comparison with time plan:

The planned work was completed in substance, but the time distribution changed. More time than expected had to be spent on endpoint testing, while less time than planned remained for frontend services. The successful endpoint verification nevertheless made the frontend service implementation easier afterwards.

7. Thursday, 16.04.2026

Planned work for today was to:

- Implement frontend user interface (4h)
- Implement frontend functionalities (4h)

Today I mainly focused on working on the frontend. I first implemented crucial frontend functionalities like routing, environment configuration and structure. I then moved on to the implementation of the planned user interface. Due to the lack of experience with this in Angular, I struggled a little bit. During the meeting with my supervisor, he advised me to let a generative AI generate a UI skeleton, instead of coding it manually myself to not waste too much time on this. At the end it still took me the rest of the day, but a UI skeleton had been able to be created with the help of generative AI.

I also had the expert meeting today and received crucial tips from him, mainly about how I should update my time plan everyday and include it when documenting the work journal, how I should document the realisation phase more in the form of a report rather than an instruction, and how I should structure my documentation.

Achieved goals:

Expert meeting completed; Frontend UI skeleton created with the help of generative AI; Crucial frontend functionalities implemented

Problems:

My lack of experience in Angular UI implementation slowed down the frontend work significantly and caused a higher time effort than originally planned. After discussion with my supervisor, I decided to use generative AI to create the initial UI skeleton as a starting point.

Support used:

Supervisor feedback; Expert feedback; Generative AI for UI skeleton generation; Angular documentation where needed

Comparison with time plan:

The workday remained focused on the planned frontend implementation. However, more effort than expected was required for the UI part, which reduced the time available for further frontend

functionality work. The use of generative AI helped to keep the implementation moving despite this delay.

8. Friday, 17.04.2026

Planned work for today was to:

- Do frontend tests (4h)
- Do backend tests (4h)

Today I had to put testing aside for now because the frontend implementation had not yet finished. The backend was mostly done, the frontend logic layer as well, but I struggled with the user interface implementation due to inexperience. After having the UI skeleton done, I stumbled across a fatal problem, which is that I found out Stooq had barricaded their stock data with CAPTCHA, which means that all automatic systems, including my backend, won't be able to retrieve data from it. So I had to consult my supervisor again. He advised me to skip integrating live data into the application, but rather to use a set of mock data stored in the database.

I managed to bring the application to a presentable stage and continued the documentation.

Achieved goals:

Running and presentable application with mock data; Documentation refined

Missed goals:

Frontend tests; Backend tests

Problems:

Testing could not be carried out as planned because the frontend was not yet finished. In addition, the originally planned stock data source became unusable for automated access due to the newly introduced CAPTCHA protection.

Support used:

Supervisor feedback; Previous implementation work; Documentation updates

Comparison with time plan:

The workday did not follow the original testing plan. Instead, time had to be redirected to risk handling and scope adaptation after the stock data source became unusable. As a result, the focus shifted from testing to stabilising the application and preparing a workable fallback solution.

9. Tuesday, 21.04.2026

Planned work for today was to:

- Document "realisation" and "control" section (4h)
- Compare realisation against requirements (4h)

Today I turned my focus onto the documentation. At this stage, the application had already reached a presentable stage, but the reporting and formalities still needed to be completed in order to match the requirements.

A major part of the day was used to write and improve the Realise section. In particular, I documented the backend, frontend, database, and containerisation aspects in a more report-like style instead of describing them as step-by-step instructions. I also added more structured explanations for implementation adjustments such as the switch from live stock data to backend-generated mock data. Furthermore, I continued the documentation of the control section, mainly about the manual testing I had done earlier. This included describing the endpoint verifications.

The second planned task, comparing the realised solution against the requirements, could only be completed partially. Although the documentation was improved significantly, some sections were still unfinished by the end of the day and required further work.

Achieved goals:

Realise section expanded and refined; Control section expanded; Documentation structure improved further; Sources section expanded; Glossary section expanded

Missed goals:

Full comparison of the implemented solution against all requirements was not yet finished; Some documentation sections remained incomplete

Problems:

A large amount of documentation still had to be completed late in the IPA. This reduced the time available for final polishing and made it necessary to prioritise the most relevant and score-critical sections first.

Comparison with time plan:

The day stayed within the planned focus on documentation, but more time than expected was required for restructuring and refining the report. As a result, the comparison of the realised solution against the requirements could not yet be completed fully and had to be continued on the following day.

10. Wednesday, 22.04.2026

Planned work for today was to:

- Evaluate planned objectives against achieved objectives (4h)
- Document "evaluation" section (2h)
- Finalise and hand in project (2h)

Today was mainly about finalising the project and adding the finishing touches to the documentation. I started with documenting the “evaluate” section of the document, comparing the achieved results with the defined project requirements. After that, I wrote my personal conclusions about this project, taking into account the things that went well as well as the things that didn’t go ideally.

In addition, I reviewed the entire document regarding project criteria and requirements and eliminated the sections that were previously still open. At the end, the document has been expanded to a satisfying stage.

Achieved goals:

Application brought to a presentable stage; Documentation finalised; list of sources and other formalities expanded; Valuable inputs provided by my supervisor on topics where I requested an opinion

Missed goals:

None

Problems:

Some formalities regarding the documentation were unclear but supervisor was able to clarify

Support used:

Supervisor opinion about documentation formalities

Comparison with time plan:

The day proceeded exactly as planned, and the hand ins were made ready for hand-in on time

15. Personal conclusion

This IPA has exposed some of my clear strengths and weaknesses where I absolutely need to improve. One major challenge I faced was during the implementation, where I was initially too optimistic about my Angular skills and underestimated how much time I needed for the UI realisation. As a result, it took away valuable time I would have needed for testing.

Probably the biggest challenge I faced by far was the documentation of this project. I had not expected to have to put in as much effort and time as I did into it. I also realised that my time management needs significant improvement. This became especially exposed when I had to redo the time planning, and too much time was consumed by the implementation and documentation, to an extent that I had to abandon some planned testing.

A further difficulty was the structured and systematic use of the IPERKA methodology. Before this project, I was not very familiar with applying IPERKA to a complete project from start to finish. I also lacked prior knowledge of decision methodologies and had to research suitable methods and examples before I could document the decision phase that would satisfy the criteria of this project.

At the same time, several parts of the project also went well. The planning phase progressed smoothly and was documented thoroughly. The mock-up design phase worked quite well too and the created UI variants received some positive feedback from colleagues, although I do see room for improvement as well. On the backend side, the implementation went comparatively well because of my longer experience with Spring Boot. In particular, the database integration through Spring Boot worked better than expected, and I was able to pass off the responsibility of structuring the database to the Spring Boot application entirely.

I am also satisfied with the final result of the project. Even though the original idea of integrating live stock data had to be abolished, I still managed to produce a presentable and working application with backend-generated mock data.

If I were to carry out a similar project again, I would create a more realistic time plan from the beginning, reserve more time for documentation and testing, and access my Angular skills properly. I would also define fallback options for risky external dependencies like the Stooq provided data source. Overall, I view this IPA as a valuable learning experience. It exposed my weaknesses, which is uncomfortable but very crucial for my career development.

Part 2: Project

1. Short summary

1. Initial situation

The goal of this IPA was to develop an application called Equity Insight. The application was designed to allow its users to keep track of stock prices, supported by features of watchlist management, stock visualisation and commenting. The program was implemented as a local, single-user application without authentication, because all relevant data are publicly available.

2. Objective

The objective was to implement a minimum viable product (MVP) that allows users to create, edit, view, and delete watchlists, search and manage stocks, display stock price development in chart form, leave comments for stocks, and switch between light mode and dark mode during runtime.

3. Implementation

The project was developed according to the IPERKA method. During planning phase, different variants of the data model, frontend structure, backend structure, user interface, chart library, and stock data source were analysed and documented. The Implementation was divided into frontend, backend and database components. The backend provides CRUD functions for watchlists, stocks and comments, while the frontend displays the user interface divided by watchlists, stocks and comments. Due to a late technical problem with the originally intended external data source, the final implementation uses a backend-generated mock stock data for chart visualisation instead of live external market data.

4. Result

The result of this IPA is a working and presentable stable minimum viable product of the application Equity Insight. The implemented solution presents the project's core functions in a way that is both easy to test and simple to understand. Specifically, the application supports watchlist management, stock selection, chart visualization using mock data, comment handling, and dynamic theme switching during runtime. In addition, the project includes documentation of the key design decisions and the overall implementation process. It also explains how technical risks were identified, assessed, and addressed throughout development.

2. Project description

The project Equity Insight is an individual practical work in the area of application development. Its goal is to create a local application for one single user to monitor and organise stock related information. The application includes watchlist management, stock visualisation and personal commenting.

The application is made for one local user only. Because of this, authentication, authorisation, and support for multiple users are not included in this project. All data entered by the user is stored locally. The focus of the project is a minimum viable product that can be completed within the given IPA time frame.

From a functional perspective, the application should allow users to manage watchlists, select stocks, view visualised stocks price history, leave comments for individual stocks, and switch between light mode and dark mode in runtime. The project also includes backend validation, a full-stack structure, and understandable, comprehensive documentation. The project is organised using the IPERKA methodology.

3. IPERKA

IPERKA is a structured, six-step project implementation method used to complete tasks in a structured, systematic way. It can be applied to a wide range of projects, including those in teams or also individually. This method separates the project process into six independent steps:

Inform (Informieren):

The task is analyzed and clearly understood. Relevant information is gathered, organized, and evaluated. Objectives, requirements, constraints, deadlines, and responsibilities are clarified.

Plan (Planen):

Possible solution approaches are developed. The task is divided into manageable steps, a timeline is established, resources are allocated, and milestones as well as quality criteria are defined.

Decide (Entscheiden):

An appropriate solution is selected based on defined criteria such as feasibility, efficiency, and available resources. Decisions are made in coordination with stakeholders where necessary.

Implement (Realisieren):

The planned solution is executed according to the defined steps. Progress is monitored, adjustments are made if required, and the process is documented.

Control (Kontrollieren):

The completed work is reviewed and tested. Results are compared with the initial plan and requirements to ensure quality and correctness. Any deviations are identified and corrected.

Evaluate (Auswerten):

The entire process and outcome are reflected upon. Strengths, weaknesses, and encountered problems are analyzed, and lessons learned are identified to support continuous improvement.

In summary, following IPERKA enforces a logical progression from understanding a task to evaluating its outcome, which promotes work structure and skill development.

1. Inform (Informieren)

In this project, I will be developing an application that provides a comprehensive interface for its users to track their stock investments. The application shall include the features of watchlists, stock searching, stock price visualisation, and commenting. Authentication and authorisation shall not be implemented for this application. The application shall be single user based. The application shall be developed with Angular and Typescript for its frontend, Spring Boot and Java for its backend and PostgreSQL for its database. Containerisation shall be done with the help of the Podman application. The application shall be tested thoroughly with Angular component tests, JUnit tests, integration tests, as well as end-to-end tests. The database shall be run in a Podman container. The backend application shall be packaged and executed as a Podman container. The frontend application shall be accessed through a web browser and started locally. The application shall not include real-time stock data tracking, notification functionality,

authentication, multi-user functionality, financial transaction functionality, or mobile device support.

1. Expected outcome

- Fully functional app with no runtime error
- Watchlist functionality that allows users to group selected stocks, watchlists will be saved on local database
- Stock searching functionality that allows users to type in a stock's company name or ticker symbol to find it
- Stock price visualisation functionality that displays a stock's price data as a graph
- Commenting system that allows users to create, edit and delete comments for an individual stock
- Toggling functionality between light mode and dark mode

2. Emerged uncertainties

- Stock searching my company name might be too difficult due to Stooq only offering data based on ticker symbol
- Interval of data displayed not yet defined for long time spans of stock prices

2. Plan (Planen)

1. Data concept

Initial data model concept:

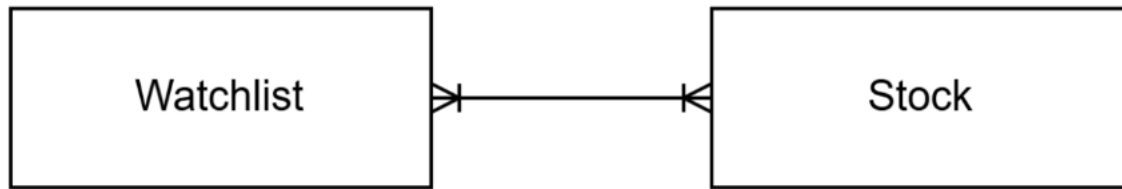


Figure 2 - ERD V1

The first data model was a simple, many mandatory to many mandatory relationship. The thought behind was just as simple, based on the logics of:

- A stock must belong to a watchlist, otherwise its info won't be saved by the database
- A stock could potentially belong to multiple watchlists
- A watchlist must contain at least one stock
- A watchlist could contain more than one stock

First revision:



Figure 3 - ERD V2

While drafting the frontend application structure, I realised improvements could be made to the initial data model.

First thing I reconsidered was whether to make “comment” a separate entity rather than leave it as simply a field under the “Stock” entity. Requirement states that a comment must include a timestamp. Based on that, I decided it was best to make an entity out of it and have it have a many optional to one relationship with the stock entity.

Second thing I reconsidered was the many mandatory to many mandatory relationship between the watchlist entity and stock entity, which could potentially cause problems during the creation of a comment or stock object. One such example is how an empty watchlist simply could not exist. By allowing the existence of an empty watchlist, I hope to improve user experience. Additionally, if a stock with comments is strictly tied to a watchlist, and that said watchlist gets

deleted, then the comments would be deleted as well. Ideally though, the comments should remain. For these reasons, I changed the relationship between the watchlist entity and stock entity to be many optional to many optional.

Second revision:

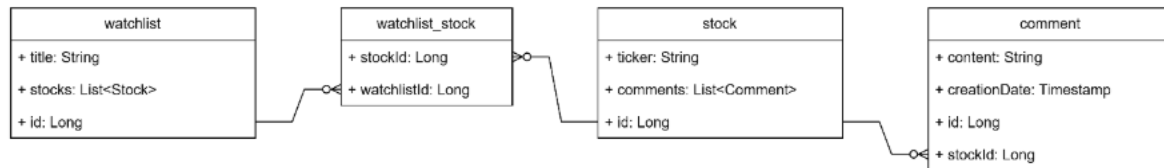


Figure 4 - ERD V3

After consulting my supervisor, he pointed out two inconsistencies about my second version of the data model.

First inconsistency was that the fields for the entities were missing. Shortly after, I added the necessary fields to all entities.

Second inconsistency was the many optional to many optional relationship. To avoid storing too much data in a single field, as well as reduce duplicating data, it is best to be converted into a join table.

Tool used: draw.io

2. Frontend structure

Initial application structure concept:

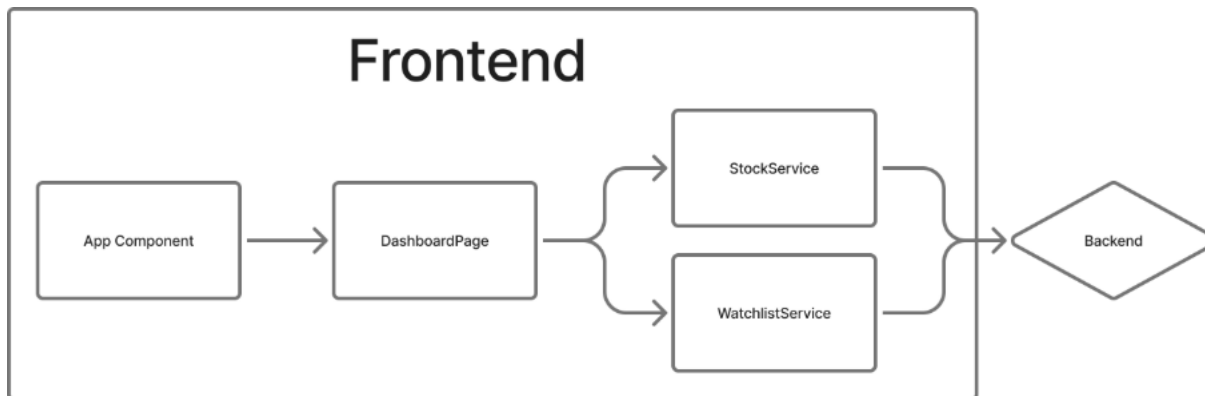


Figure 5 - frontend structure V1

The initial idea was to create a single page application, because it would be sufficient to cover the requirements. This was created based on the initial backend application structure concept, which in itself was based on the initial data model concept, in which “comment” was still a field under the stock entity, rather than being its own entity.

It features an “App Component”, which is the standard root component of an Angular application. The App Component would then call up a “DashboardPage” that represents the entire user interface. The two services, divided by entities, would help the DashboardPage fetch data from the Backend.

First revision:

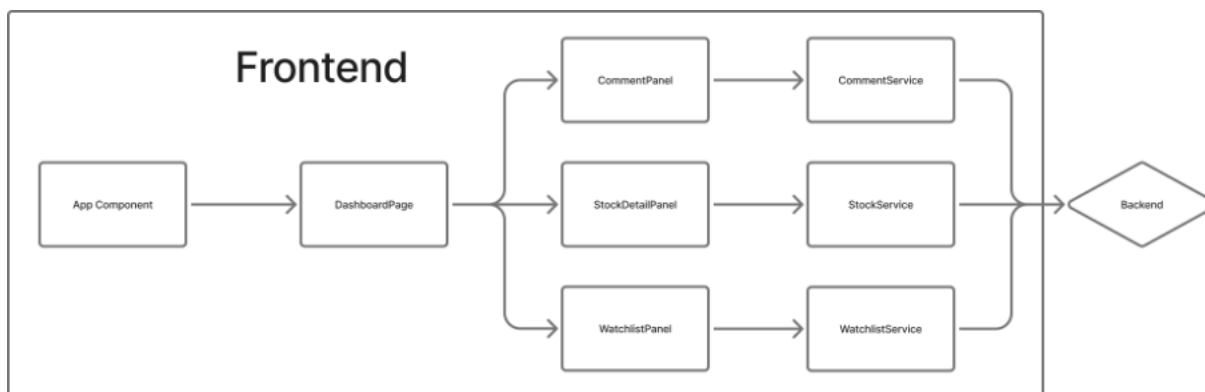


Figure 6 - frontend structure V2

After drafting the two user interface mock-up variants, I noticed the single DashboardPage might be taking on too many responsibilities. After deciding to make “comment” an independent entity, I have also split the DashboardPage into three “panels”, making each entity to have its own panel and its own service class.

3. Backend structure

Initial application structure concept:

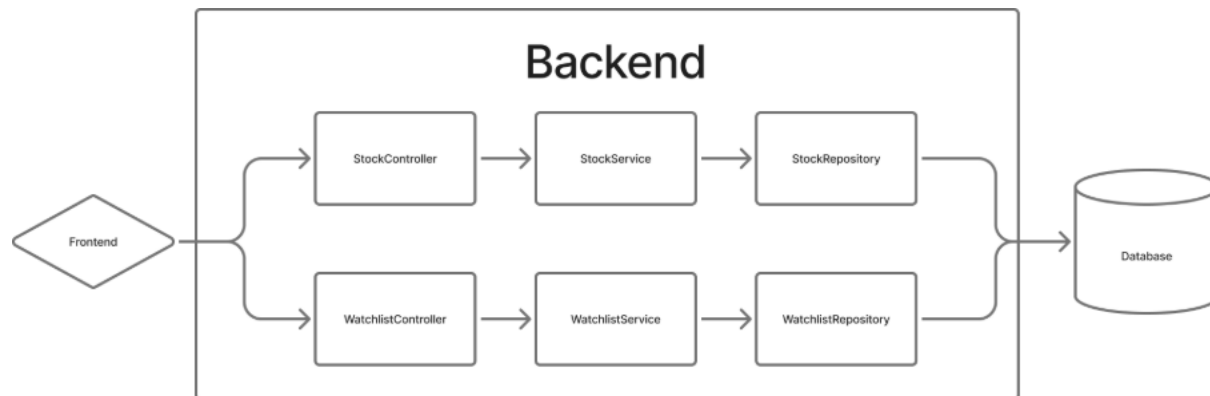


Figure 7 - backend structure V1

This was created based on the initial data model concept. It includes all three Spring Boot application layers (controller + service + repository) for both entities.

First revision:

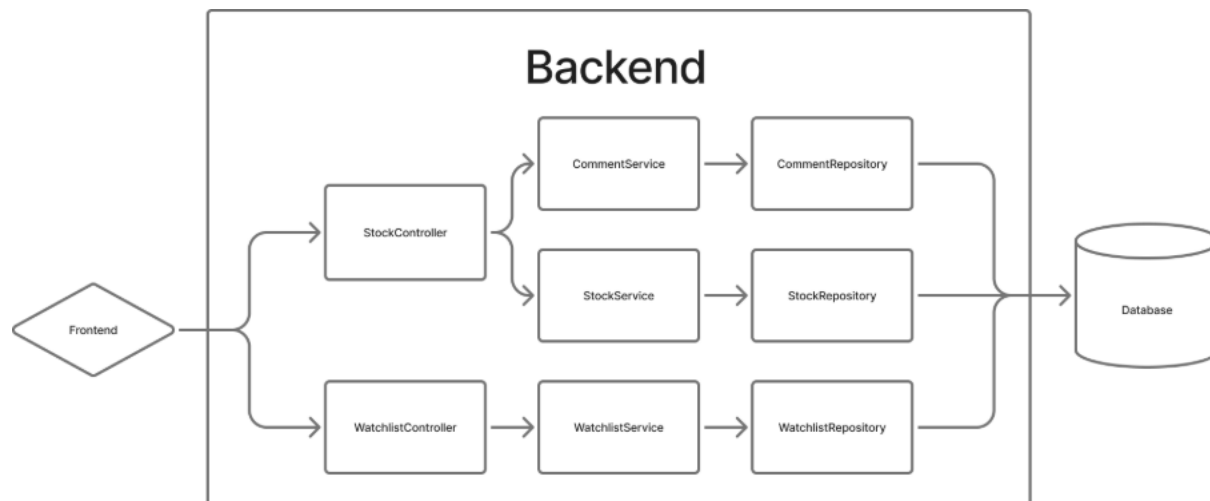


Figure 8 - backend structure V2

I revised the backend structure concept after deciding to make an independent comment entity. In this version, I added a service and a repository for the comment entity, however still let the 'StockController' handle requests for the comment module, based on the fact that comments are always tied to a stock.

Second revision:

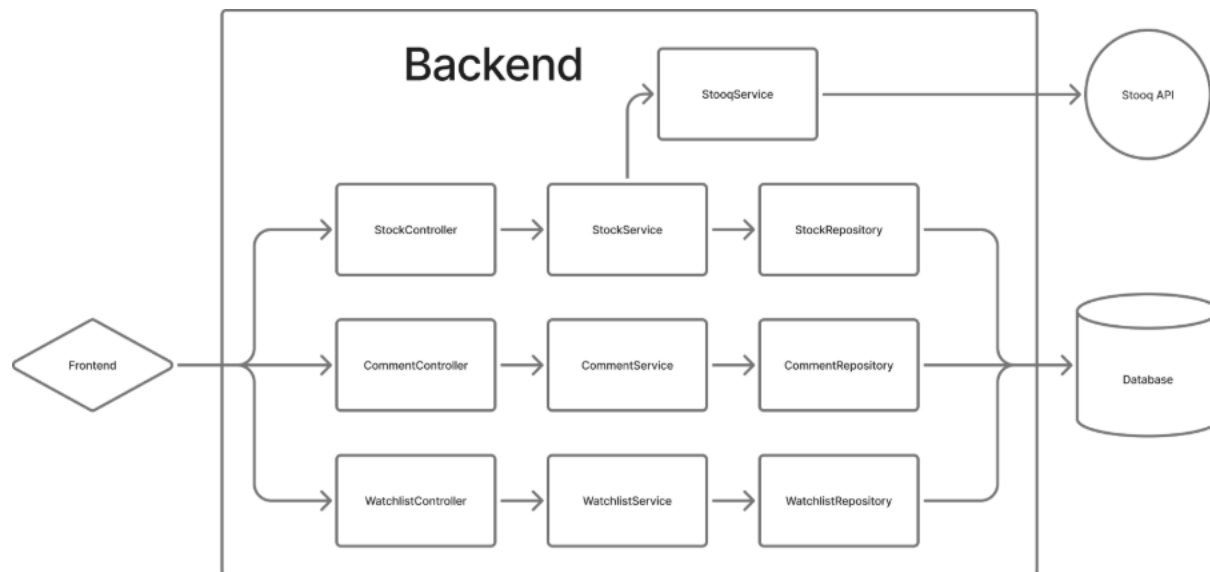


Figure 9 - backend structure V3

In this version, I decided to still split the responsibilities of the StockController and give the comment entity its own controller, in order to allow an easier future expansion. Additionally, I added a separate StooqService to handle fetching data from an external data source that could be volatile or unstable, mainly to aid in error handling.

Third revision:

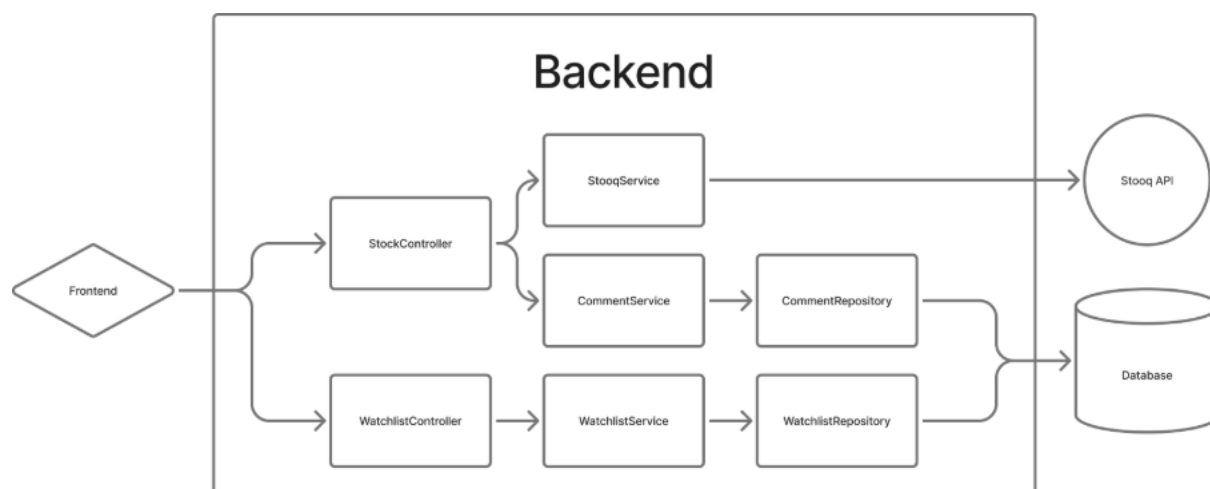


Figure 10 - backend structure V4

While drafting backend API endpoints, I realised that I actually don't necessarily have to store independent stock entities, and instead could have its fields split up and stored in the watchlist entity and comment entity instead. I would say that this version might not necessarily be better than the last one, but this would add another option when I am at the decision phase of the project.

4. User interface mock-ups

To get a more comprehensive overview of the user interface, and have a better basis for the implementation other than just my own imagination, I have created one set of initial mock-ups, and then based on that, created two variants of improved sets of mock-ups. The purpose of this was to visualise the application before the implementation stage, in order to compare different layout ideas, and verify whether it matches the requirements. For this, I have chosen to use the tool Figma in aid. I will document the final design choice in the “decide” section of this document.

The mock-ups were not only used for visualising designs, but also as a tool for validating usability and technical feasibility. In particular, it helped me think of the workflow of the application. In addition, I have gathered feedbacks from a few colleagues about the mock-ups, and adjusted the implementation accordingly.

Tool used: Figma

Initial brainstorm mock-up:

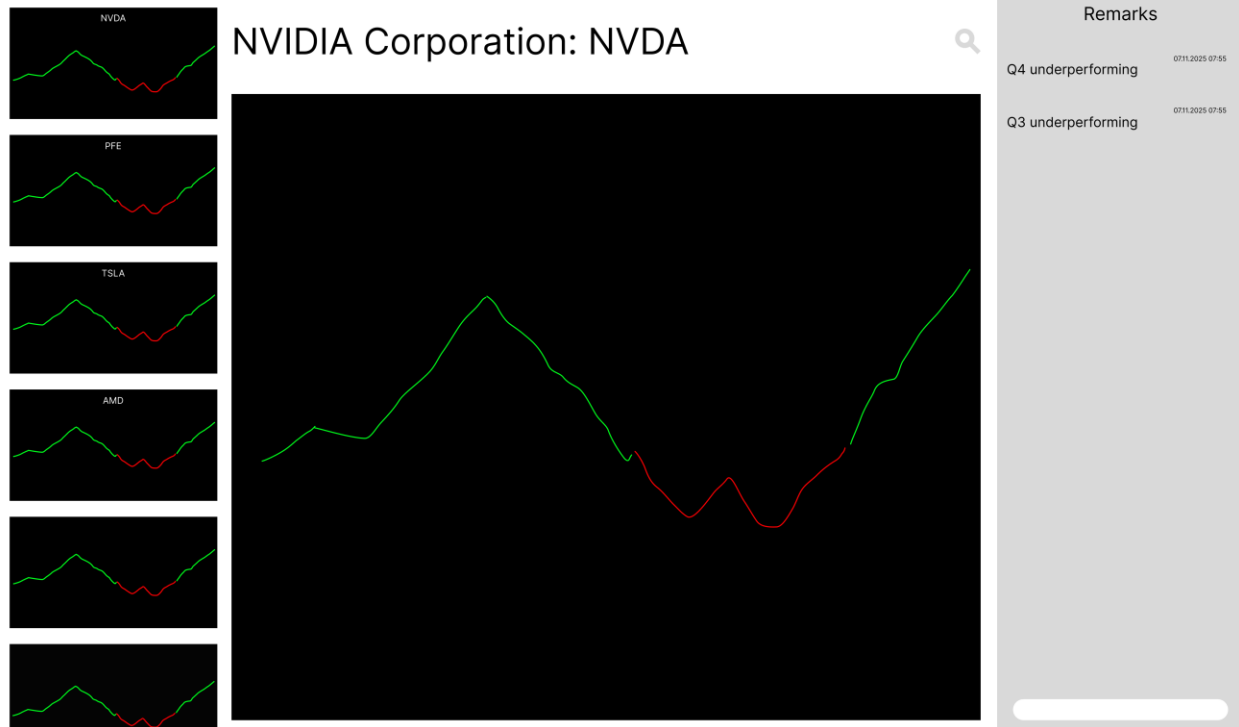


Figure 11 - dashboard mock-up V1

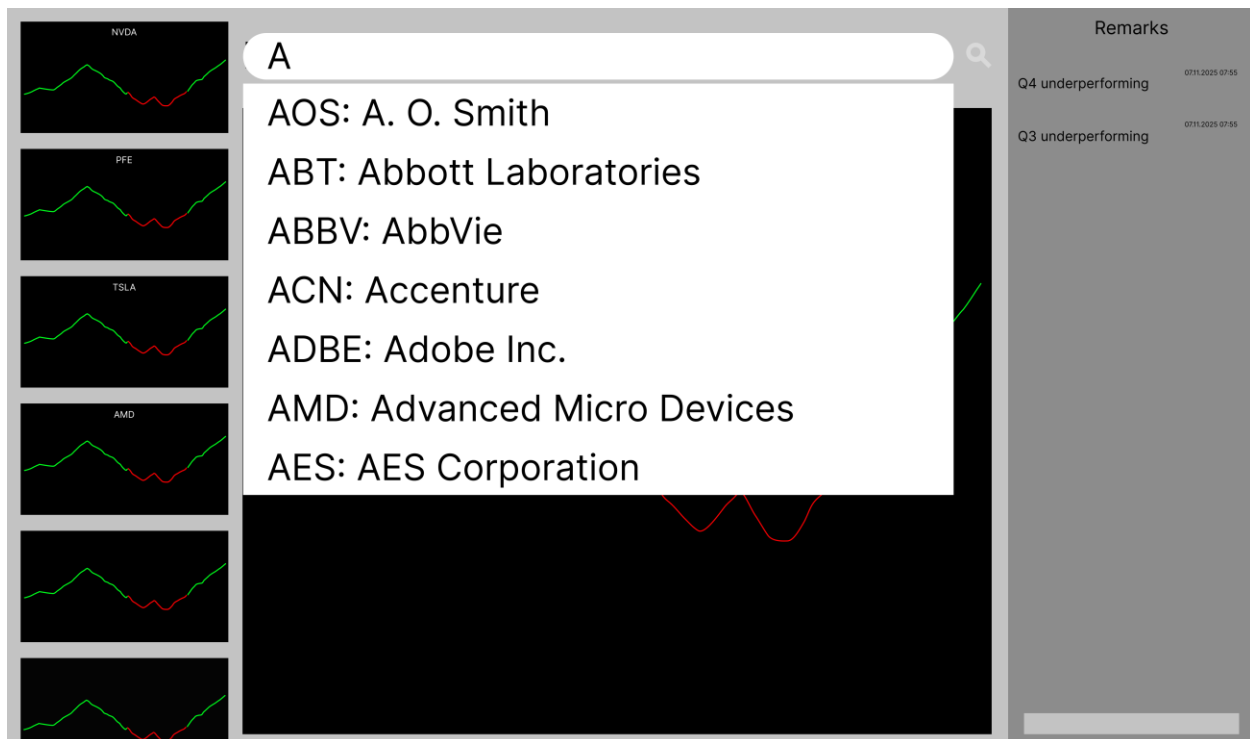


Figure 12 - stock search mock-up V1

This set of mock-ups was created as preliminary work of this project. It was used to check the general feasibility of the implementation of this application. At this point, the goal was not yet to finalise a design, but rather to vaguely identify the main user interface areas.

This version already showcases the separation of the dashboard page into three separate sections:

- Watchlist
- Stock graph
- Comment

This helped me realise that the user interface should not be built as a single overloaded page, but it should instead be structured into multiple sections for a division of responsibilities.

New mock-ups based on requirements, vertical scrolling variant:

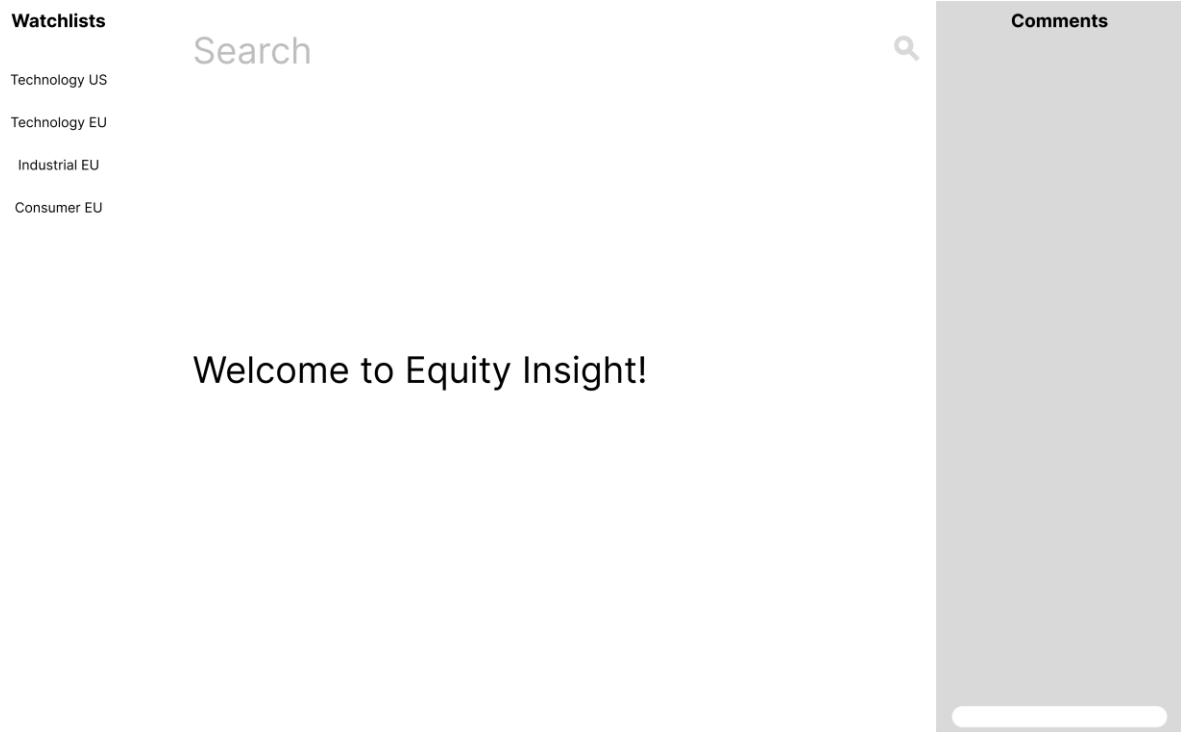
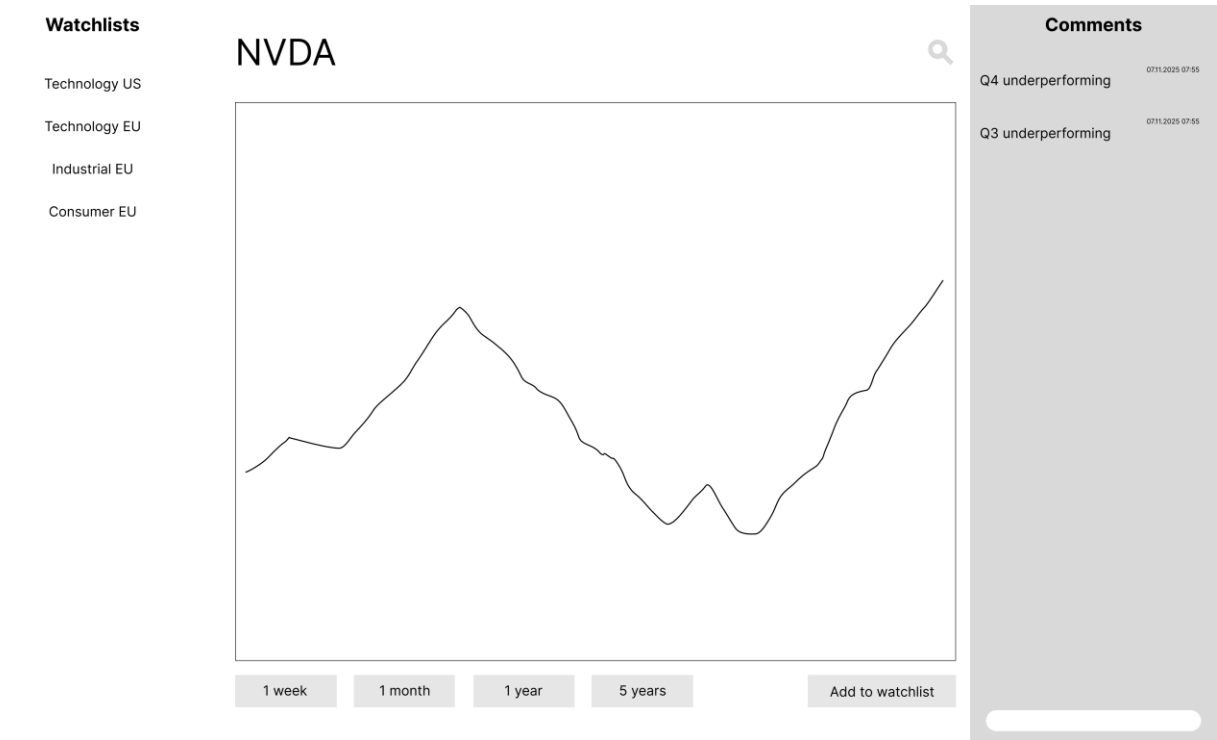


Figure 13 - dashboard mock-up V2



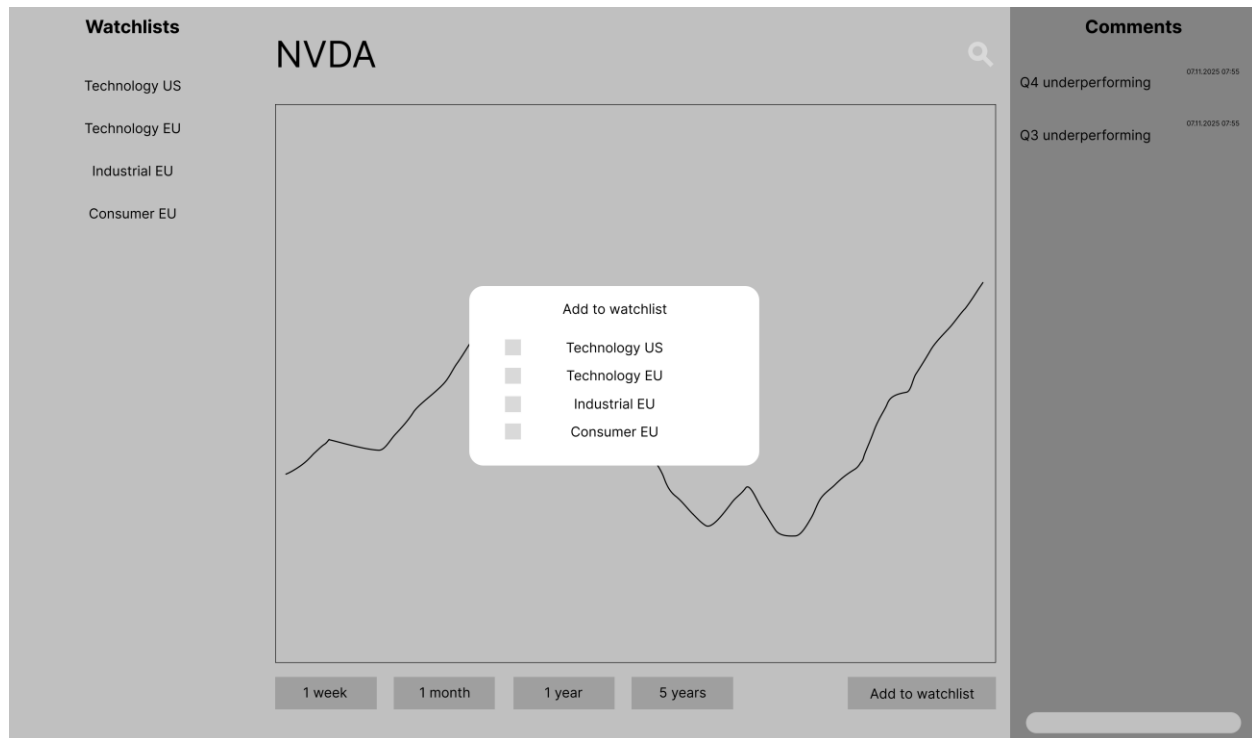


Figure 16 - add to watchlist dialog mock-up V2

This first, detailed design variant was made with the emphasis of vertical scrolling elements. In this version, the user interface is designed in a way that display listed items in a natural top-to-bottom order. This variant appeared to be more practical for this application because of this. Two out of the three main sections of the dashboard page display listed items. According to the gathered feedbacks from my colleagues, this also seems to be aligned with common user expectations.

New mock-ups based on requirements, horizontal scrolling variant:

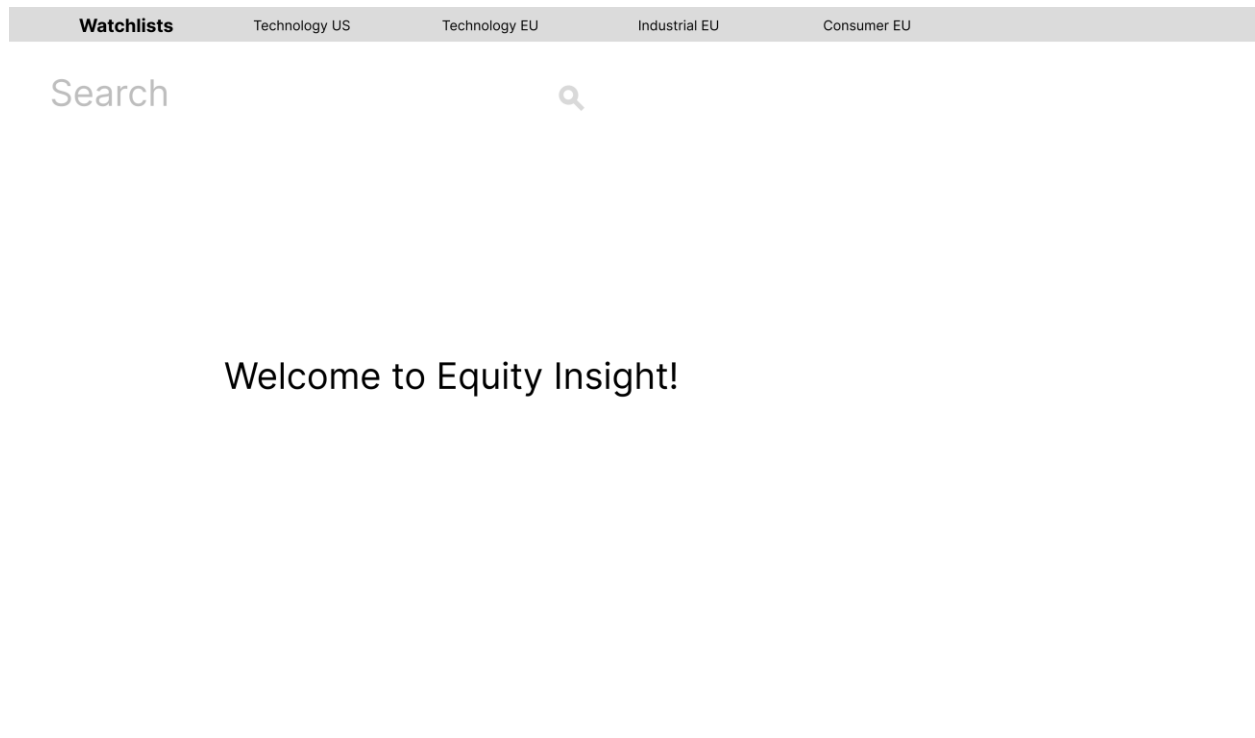


Figure 17 - dashboard mock-up V3

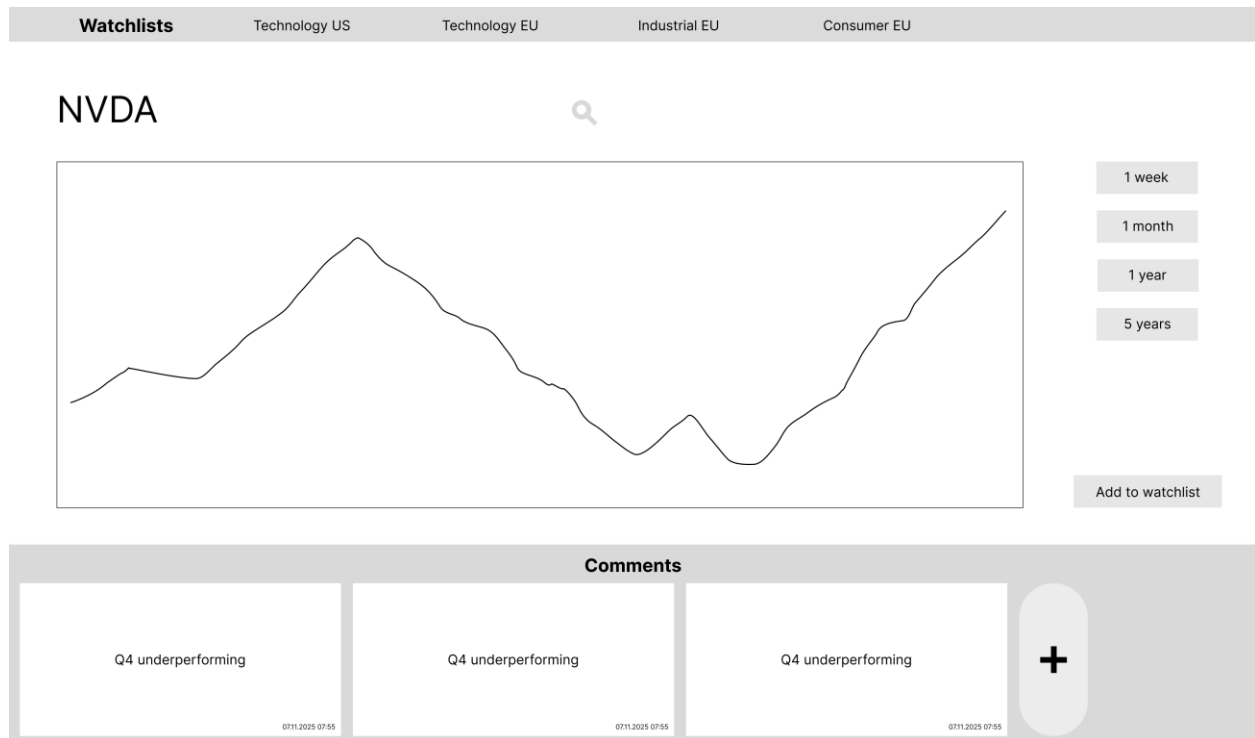


Figure 18 - stock view mock-up V3

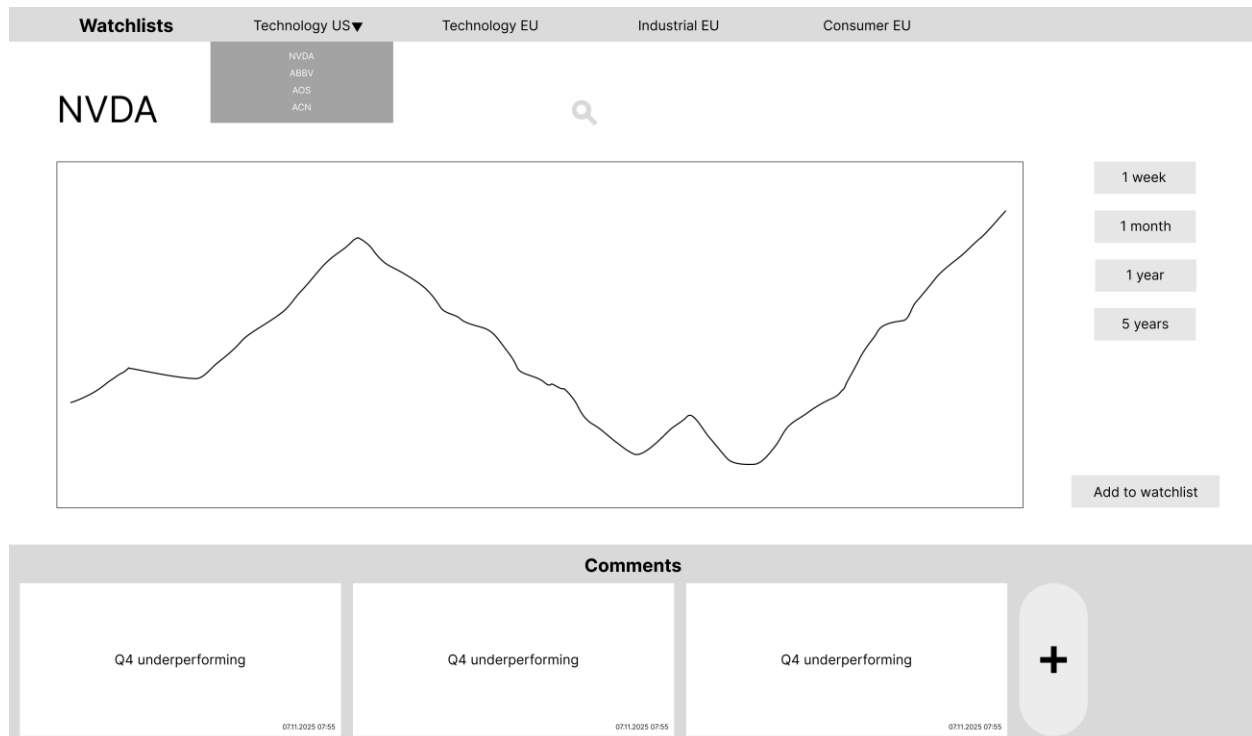


Figure 19 - watchlist view mock-up V3

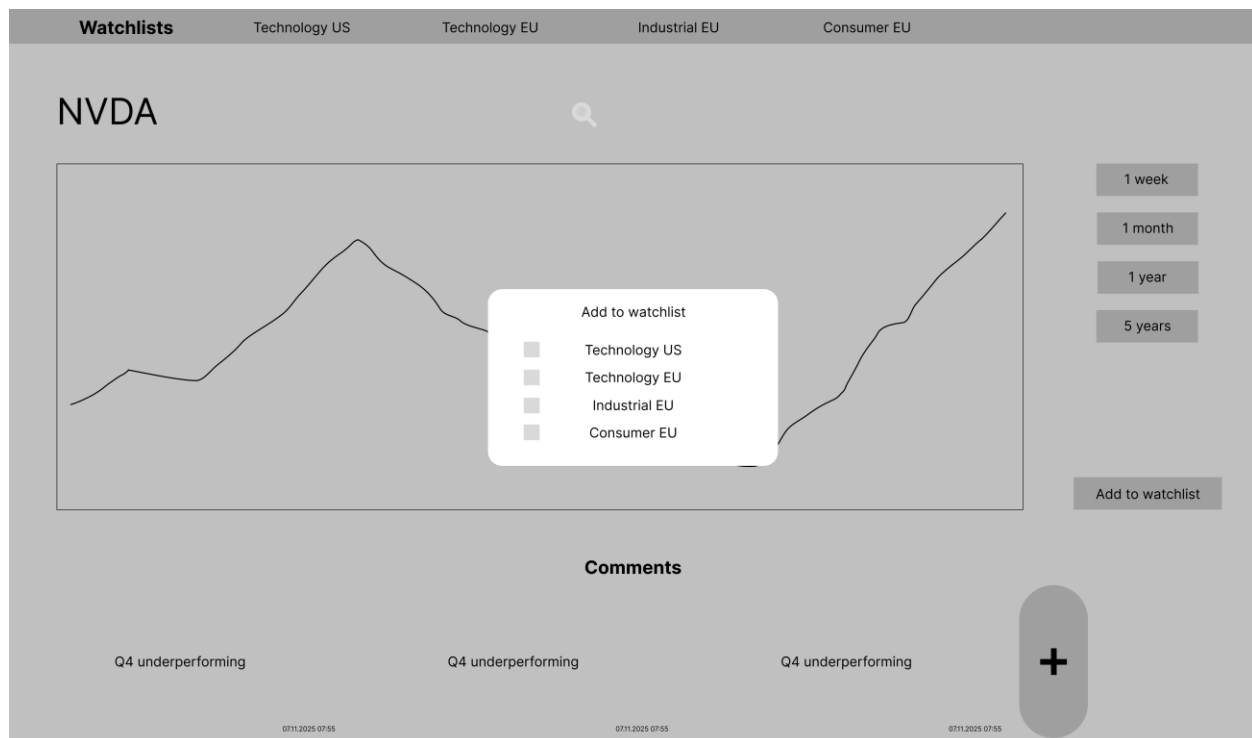


Figure 20 - add to watchlist dialog mock-up V3

This second variant was made with more of an emphasis on horizontal scrolling elements. From a visual perspective, this version appeared more attractive and modern to me. However, after

further analysis and colleague feedbacks, it became clear that horizontal display is less natural for displaying listed items. Furthermore, since the vast majority of web pages emphasise on vertical scrolling, choosing a horizontal approach would introduce more uncertainty and complexity in the future, without offering a functional advantage.

As part of the prototyping and validation of the user interface, I have requested feedbacks from a few colleagues in an email for each UI variant:

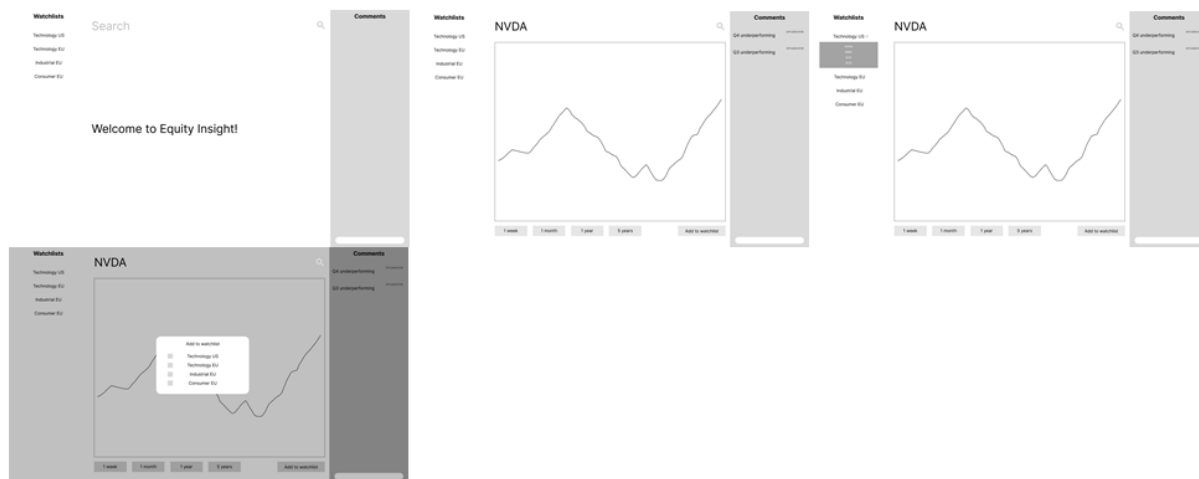
Feedback on UI mock-ups for my IPA



Dear colleagues

I am currently working on my IPA "Equity Insight", and I would like to get some quick feedback on my mock-ups. The application is a single-user stock tracking app.

First set of mock-ups:



Thanks in advance,
Best regards,



Masa Lam

Figure 21 - feedback request for V2

ML Lam, Masa
To [redacted], [redacted], [redacted], [redacted]

😊 Reply

Dear colleagues

I am currently working on my IPA "Equity Insight", and I would like to get some quick feedback on my mock-ups. The application is a single-user stock tracking app.

Second set of mock-ups:



Thanks in advance,
Best regards,



Masa Lam

Figure 22 - feedback request for V3

For privacy reasons, the names of the recipients were intentionally hidden. I have summarised the feedback into positives and negatives.

Positives:

- The UI is minimalistic, clean, and understandable
- The hierarchy is clear
- Comment section brings context to the graphs
- The buttons are well placed
- Not too many colours are used, which could otherwise be distracting
- The search field is big enough
- The rest of the page darkened when the dialog is open
- The page in general is comprehensible

Negatives:

- Comment section isn't specified to which graph it is related
- The colours are too dull and unexciting
- The "Welcome to Equity Insight" welcoming phrase lacks context
- A fitting logo lacks
- X and Y axes are missing
- Watchlist elements have the same background colour as the rest of the page
- Cancel and submit actions are missing in the "add to watchlist" dialog

The gathered feedbacks will be heavily considered during the implementation phase. Due to timing constraints and it offering too little advantages, no new user interface mock-ups will be created to replace the current ones.

5. Backend API endpoint definition:

The API endpoints for the backend were defined based on the requirements driven by the UI designs and CRUD structure for the main entities of the application.

Below is the exact list of endpoints displayed by the Swagger UI:

Servers

http://localhost:8080 - Generated server url

Stocks Stock price history endpoints		^
GET	/stocks/{ticker}/prices	Get stock price history
Comments Comment management endpoints		^
POST	/comments	Create a comment
DELETE	/comments/{id}	Delete a comment
PATCH	/comments/{id}	Update a comment
GET	/comments/stock/{stockId}	Get comments for a stock
Watchlists Watchlist management endpoints		^
GET	/watchlists	Get all watchlists
POST	/watchlists	Create a watchlist
GET	/watchlists/{id}/stocks	Get stocks in a watchlist
POST	/watchlists/{id}/stocks	Add a stock to a watchlist
DELETE	/watchlists/{id}	Delete a watchlist
PATCH	/watchlists/{id}	Update a watchlist title
DELETE	/watchlists/{watchlistId}/stocks/{stockId}	Remove a stock from a watchlist

Figure 23 - Swagger endpoint list

6. Implementation concept

The application will be implemented as a web application. It will have a clear separation between the frontend stack, backend stack and the database.

The frontend will be developed with Angular and TypeScript. The frontend will be responsible for displaying the user interface, receiving user input and sending requests to the backend.

The backend will be developed with Java and Spring Boot. It will be responsible for processing requests, validating input, and for fetching data from an external data source for market data.

The database will be setup as a PostgreSQL database and ran in a Podman container.

The planned data flow would look as follows: The user interacts with the frontend Angular application, the frontend sends requests to the backend Spring Boot application as HTTP requests and received by REST endpoints, then the backend application processes the requests and handles them by reading from, altering the PostgreSQL database, or by accessing market data from third party market data APIs. The frontend would then display the data in a way that is easy to understand, for example in charts or watchlist details.

7. Test concept

Tests will be created for the project in order to verify the correctness of the implementation of the requirements, as well as to make sure the different layers of the application work together properly.

Testing will be done on different levels. On the frontend, testing will be done with the help of built-in Angular component tests. On the backend, unit tests will be used to test service logic, and integration tests will be used to test REST endpoints and database access. Manual end-to-end tests will also be carried out to test user flows.

These areas are planned to be tested:

- Creation, editing, viewing and deletion of watchlists
- Stock search
- Retrieval and display of stock data
- Creation, editing and deletion of comments
- Light / dark mode toggle

Test cases:

- Create watchlist
Create a new watchlist with a valid title.
Expected result: The watchlist is created and appears in the watchlist list.
- Create watchlist with empty title
Try to create a watchlist without entering a title.
Expected result: The watchlist is not created and a validation error is shown.

- **Rename watchlist**
Change the title of an existing watchlist.
Expected result: The updated title is saved and displayed correctly.
- **Delete watchlist**
Delete an existing watchlist.
Expected result: The watchlist is removed from the list.
- **Add stock to watchlist with valid ticker**
Add a valid ticker symbol to a selected watchlist.
Expected result: The stock is added to the watchlist and displayed in the stock list.
- **Add duplicate stock to watchlist**
Try to add the same ticker twice to the same watchlist.
Expected result: The stock is not added a second time.
- **Add stock with invalid ticker**
Enter an invalid ticker symbol.
Expected result: The stock is not added and an error message is shown.
- **Display stock chart**
Select a stock in a watchlist.
Expected result: A chart with stock price data is displayed.
- **Switch chart time range**
Change the selected chart range, for example from 1M to 3M.
Expected result: The chart updates and displays data for the selected range.
- **Create comment**
Add a comment to a selected stock.
Expected result: The comment is saved and displayed under the selected stock.
- **Edit comment**
Edit the content of an existing comment.
Expected result: The updated comment is saved and the last edited timestamp changes.
- **Delete comment**
Delete an existing comment.
Expected result: The comment is removed from the comment list.
- **Theme toggle**
Switch from light mode to dark mode and back.
Expected result: The application theme changes correctly at runtime.
- **Persist local data**
Close and restart the application after creating watchlists and comments.
Expected result: Locally stored user data is still available after restart.
- **Request nonexistent resource**
Try to access a watchlist, stock, or comment that does not exist.
Expected result: The backend returns a controlled error response instead of crashing.

3. Decide (Entscheiden)

1. Multi-user or single-user

Question to be answered in this section: Implement a multi-user or a single-user application for this project?

Options:

- Multi-user
- Single-user

For this decision, I applied the “requirement-based decision-making” methodology. I will use a table for the visualization:

Requirement	Multi-user	Single-user
Matches project scope	no	yes
Uses local data	no	yes
Requires authentication	yes	no
Feasible in 10 days	no	yes

The comparison showed that a single-user application is more suitable for this project, based on every requirement. User-specific data such as watchlists and comments will be stored locally, which the market data itself is publicly available. Because of this, implementing a multi-user application will offer very little benefit for the enormous amount of time it would consume, which I could not afford.

Final decision: Single-user

2. Which Graph library

Question to be answered in this section: Which graph library to use to display stock data?

After searching the web as preliminary work, I found a single viable option for presenting graphs in my frontend code base. The option selected is Apache ECharts.

3. Which stock data API

Question to be answered in this section: Which external stock data sources to use for this application?

Before the start of the project, I researched different potential data sources for stock data to be used in this project. The three main options I found were:

- Alpha Vantage API
- Stooq API
- Yahoo Finance API

To make a decision and make it traceable, I will be implementing the “decision matrix” method.

Stock data source decision matrix (5pt max.):

Final score = point x weight (e.g. 2 x 30%), all four added together

Criterion	Weight	Alpha Vantage	Stooq	Yahoo Finance
Free usage	30%	2	5	5
Legally compliant	30%	5	5	3
Historical data	30%	5	5	5
Stability	10%	5	5	1
score		4.1	5	4

Justification:

Stooq is overall the best fit for this project, as it delivers free unlimited data, is legally compliant, offers historical data, and is very stable.

During the implementation process, I found out that Stooq had newly implemented CAPTCHA to block access from automated systems, exactly the kind like the backend application of this

project, whereas I assumed until that point, that Stooq remained a free and accessible data source. For this reason, I had to make a new critical decision. I found three potential alternative ways of still implementing this application:

- Either replace Stooq with an alternative data source that still fits the project requirements
- Use fixed mock stock data stored in the database using SQL injection, suggested by my supervisor
- Or have the backend generate mock data

To make this decision, I utilised the “knockout criteria” methodology. Below is the visualised decision making process:

Criterion	New data source	Fixed mock data	Generated mock data
Is implementable within the remaining time	knockout	yes	yes
Is stable for the demonstration	knockout	yes	yes
Doesn't introduce potential uncertainty (legal, technical)	knockout	yes	yes
Is believable (enough) during demonstration	yes	knockout	yes
result	knocked out thrice	knocked out once	passed

After the elimination process according to this “knockout criteria” methodology, only the third option – generated mock data, passed.

Final decision: use backend generated mock data

4. Vertical or horizontal scrolling

Question to be answered in this section: Which user interface variant to implement?

In the planning phase, I made two detailed UI variants. Now I need to decide which variant to implement.

Options:

- Variant one - UI focused on vertical scrolling
- Variant two - UI focused on horizontal scrolling

To make this decision, I used the criteria-based comparison. The UI variant will get a cross if it fits the criterion.

Criterion	Vertical	Horizontal
Suitability for list based content	x	
Usability / readability	x	x
Implementation simplicity	x	
total:	3	1

Based on the fact that vertical scrolling elements are just simpler to implement, and more suitable for list-based content.

Final decision: implement UI focused on vertical scrolling

5. Backend structure

During the planning phase, I have drafted 4 different variants of a backend Spring Boot application structure, each varying in complexity and responsibility division. Here I will make the decision of which variant to implement in the realise phase. For this, I will be using the decision matrix in aid.

Options:

- Variant one - very simple:

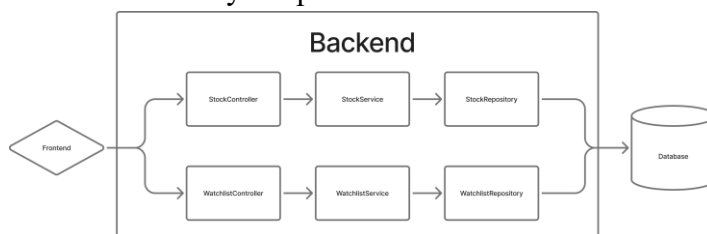


Figure 24 - backend structure V1

- Only two entities
- StockService handles fetching external data
- Stock module handles commenting logic
- Variant two - more complex:

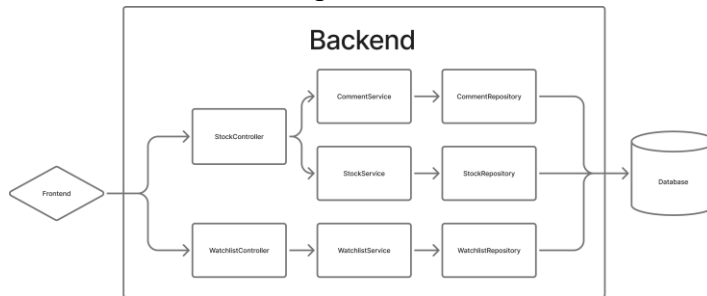


Figure 25 - backend structure V2

- Three entities
- StockController handles comment endpoints
- StockService still handles fetching external data
- Variant three - max complexity:

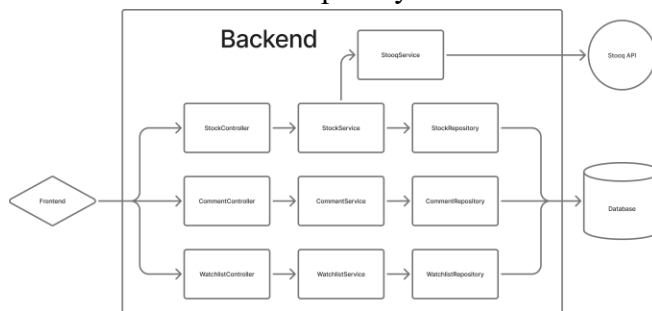


Figure 26 - backend structure V3

- Three entities
- Separate StooqService handles fetching external data
- Separate CommentController handles comment endpoints
- Variant four - hybrid:

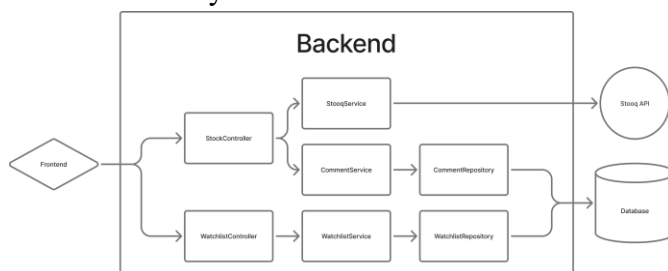


Figure 27 - backend structure V4

- Only two entities
- StockController handles stock endpoints and comment endpoints

- Separate StooqService handles fetching external data
- Fields originally in the stock entity distributed between the comment entity and watchlist entity

Backend structure decision matrix:

Equal weight, maximum five points for each criterion, final score = all points added together

Criterion	Variant one	Variant two	Variant three	Variant four
Responsibility Separation	1	3	5	3
Maintainability	1	4	5	3
Extensibility	1	4	5	3
Error handling simplicity	1	4	5	3
Implementation effort	5	3	1	4
total:	9	18	21	16

After consideration with the help of this decision matrix, the decision to implement this specific backend application with maximum complexity was made.

Final decision: implement variant three

4. Realise (Realisieren)

1. Environment and structure

For the implementation, I am using IntelliJ as the IDE. For containerisation, I am using Podman Desktop.

The final application structure:

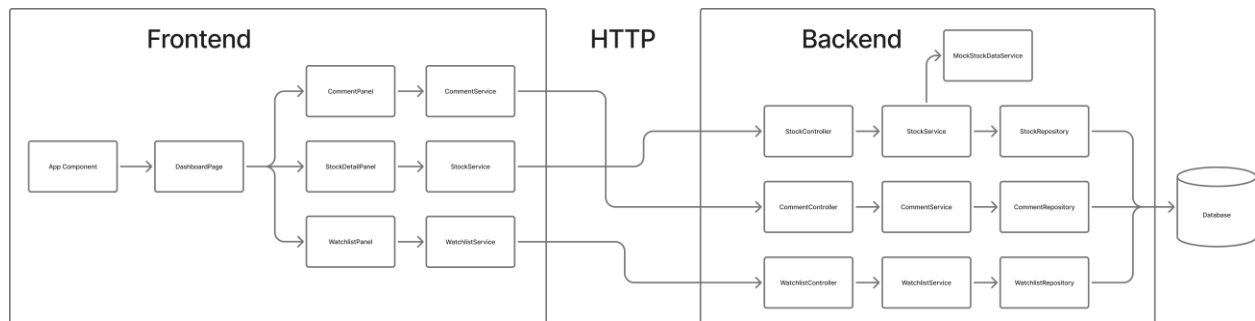


Figure 28 - application structure

Code base structure:

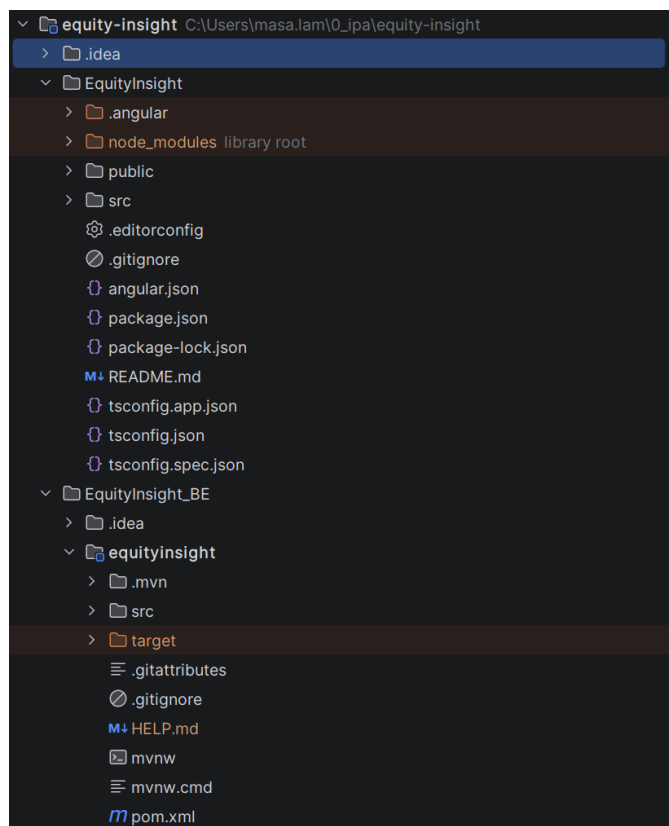


Figure 29 - code base structure

In the screenshot above you can see the structure of the monolithic repository code base.

In the folder **EquityInsight**, you will find the frontend application. The initial project skeleton was generated with Angular CLI.

In the folder `EquityInsight_BE`, you will find the `equityinsight` folder, which represents the Spring Boot backend application. This is structured this way because Spring Initializr was used to create the backend component.

2. Database

For the creation of the database, this podman command is run:

```
podman run -d --name equityinsight -postgres -e POSTGRES_DB=equityinsight -e
POSTGRES_USER=postgres -e POSTGRES_PASSWORD=postgres -p 5432:5432 -v
equityinsight -postgres-data:/var/lib/postgresql/data:Z postgres:16
```

Explanation:

<code>-d</code>	Runs container in background
<code>--name equityinsight</code>	Name of the container
<code>POSTGRES_DB=equityinsight</code>	Creates the database
<code>POSTGRES_USER=postgres</code>	Database username
<code>POSTGRES_PASSWORD=postgres</code>	Database password
<code>-p 5432:5432</code>	Exposes PostgreSQL on localhost
<code>-postgres-data:/var/lib/postgresql/data:Z</code>	Persistent storage
<code>Postgres:16</code>	Stable PostgreSQL version

The application uses PostgreSQL as its relational database. The database is responsible for storing all locally managed application data, especially watchlists, comments, and stock-related references that are required by the implemented features. PostgreSQL was chosen because it is stable, widely used, well supported by Spring Boot, and suitable for relational data models with clearly defined entity relationships.

During implementation, the database structure was derived from the previously created data model and refined where necessary. The main database entities are watchlists, comments, and stocks. Particular attention had to be paid to the relationships between these entities, because earlier design versions revealed risks such as unnecessary duplication, unsuitable dependencies, or restrictions for later CRUD operations.

The database was not installed directly on the local machine, but run inside a Podman container. This made it possible to keep the environment reproducible and separated from the rest of the local system. It also simplified the setup of the development environment and reduced the risk of inconsistencies between configuration states.

The database connection was verified early in the IPA. During the initial setup, a connection problem occurred between the backend and the PostgreSQL container. This issue was resolved by reinitialising the backend and the container configuration. After that correction, the connection could be established and used as the basis for the later implementation of repositories, services, and CRUD operations.

From an implementation perspective, the database supports the local persistence required by the project scope. Since the final solution no longer depends on live stock data integration, the database mainly serves the user-managed parts of the application, while chart data is currently provided through backend-generated mock data.

Main database responsibilities:

- storage of watchlists
- storage of comments
- storage of local stock references
- persistence of user-managed data between application runs
- support for CRUD operations through Spring Data JPA

Important implementation observations:

- The database structure had to be refined during implementation because some earlier data model variants were not practical enough.
- Entity relationships were an important implementation focus, especially for watchlists, stocks, and comments.
- Running the database in a container improved reproducibility and separation of the runtime environment.
- The backend-database connection was successfully tested after correcting the initial setup issue.

3. Backend

The backend was implemented first, because it defines the core application logic. The frontend will also be dependent on the backend logic. The implementation follows a layered Spring Boot structure, dividing logic by controllers, services and repositories. The structure was chosen in order to separate responsibilities more clearly.

The first thing I implemented were the entities and their relationships. It was based on the previously created data concept. During this work, weaknesses in the original design exposed themselves. In particular, the relationships between watchlists, stocks and comments had to be refined to better support CRUD operations and avoid duplications. The data model was implemented and corrected during the implementation phase.

After finishing the implementation of the entities, the backend modules were implemented incrementally. The watchlist and comment modules were built with their own controller, service, repository, DTO, and mapper structure. In the stock module, the original idea of using live external stock data had to be abandoned due to a late technical problem with the selected source. To keep the application functional and testable within the remaining IPA time, the backend was adapted so that stock price data is now generated as random mock data in the backend instead of being retrieved from a live external source.

Another important implementation step was the creation of DTOs and backend validation. DTOs were used to avoid exposing entire JPA entities directly and keep requests more stable and simpler.

The backend endpoints were then tested manually with the browser and with Postman. The tests were important to verify the endpoints and to avoid mismatch between the expected data and sent data. This was crucial for future integration with the frontend.

Implemented backend modules:

- Watchlist module
 - CRUD operations for watchlists
 - title updates

- backend validation for input data
- Comment module
 - create, update, and delete comments
 - comments linked to a stock
 - timestamp handling for creation and editing
- Stock module
 - local stock handling by ticker symbol
 - mock price history generation in the backend
 - chart data endpoint for frontend consumption

Implemented backend structure:

Layer	Purpose in the project
Controller	Accepts HTTP requests and returns responses
Service	Contains business logic and coordinates operations
Repository	Handles database access through Spring Data JPA
DTO	Defines stable request and response structures
Mapper	Converts between entities and DTOs
Exception	Handles invalid or missing data in a controlled way

Important implementation adjustments:

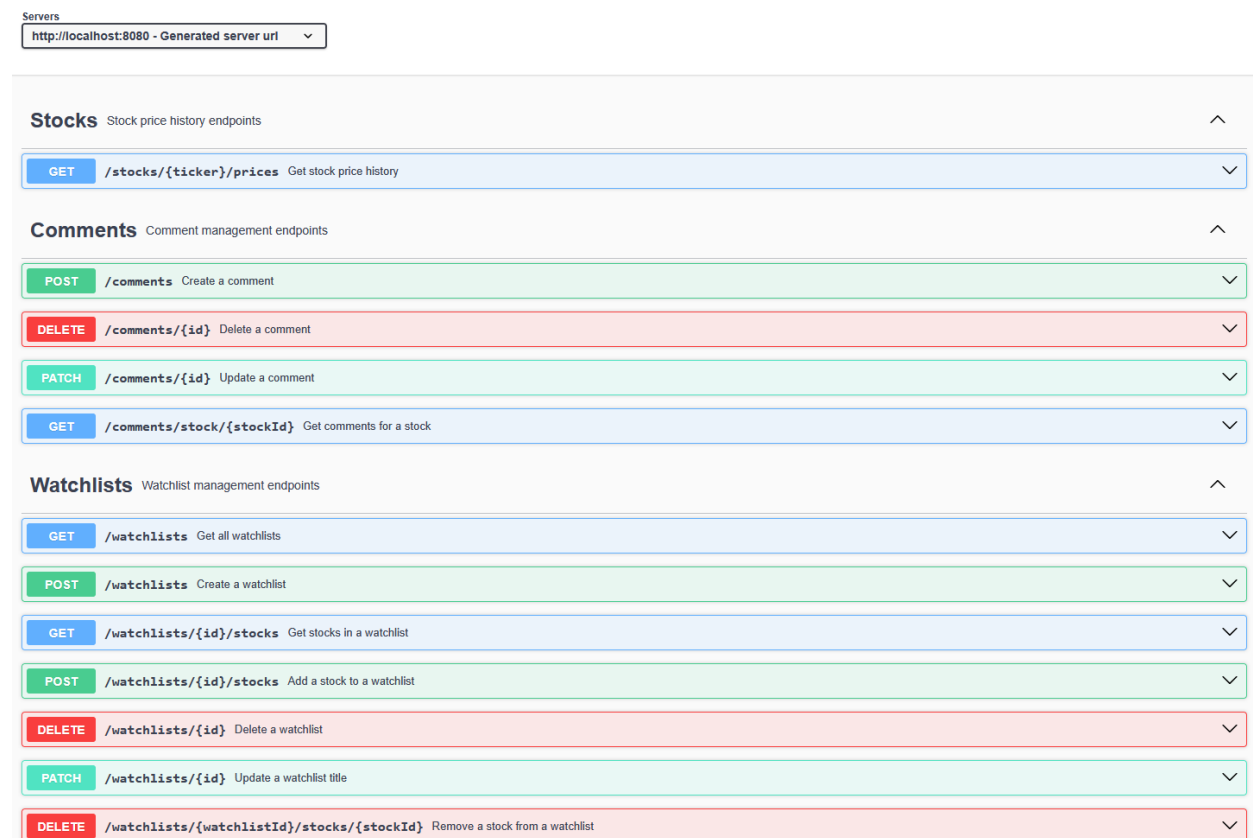
Topic	Initial idea	Final implementation
Stock market data	Live external source	Backend-generated mock data
Entity exposure	Direct entity usage possible	DTO-based API structure
Comment handling	Initially more tightly coupled	Dedicated comment module structure
Error handling	Basic implementation	Custom exceptions and validation

Main backend implementation steps

1. Implementation of entity classes and their relationships
2. Correction and refinement of the data model during coding
3. Implementation of repositories, services, and controllers
4. Introduction of DTOs and mappers
5. Addition of backend validation and custom exceptions
6. Manual testing of endpoints with browser and Postman
7. Adaptation of the stock data strategy from live data to backend-generated mock data

Swagger:

To document and inspect the backend REST endpoints, Swagger was utilised. Swagger provides an interactive overview of the implemented endpoints, including their paths, HTTP methods, request parameters, and response structures. In this project, Swagger was useful because it made the backend API easier to understand, test, and verify during implementation. It also supported the frontend integration, because the request and response structures could be checked directly in a structured and visual form.



The screenshot displays the Swagger UI interface. At the top, there is a 'Servers' section with a dropdown menu showing 'http://localhost:8080 - Generated server url'. Below this, the endpoints are organized into three main sections: 'Stocks' (Stock price history endpoints), 'Comments' (Comment management endpoints), and 'Watchlists' (Watchlist management endpoints). Each section contains a list of endpoints with their respective HTTP methods (GET, POST, DELETE, PATCH) and descriptions. For example, under 'Stocks', there is a GET endpoint for '/stocks/{ticker}/prices' to get stock price history. Under 'Comments', there are endpoints for creating, deleting, and updating comments. Under 'Watchlists', there are endpoints for getting, creating, adding stocks to, deleting, and updating watchlists.

Figure 30 - Swagger endpoint list

One Swagger endpoint documentation example:

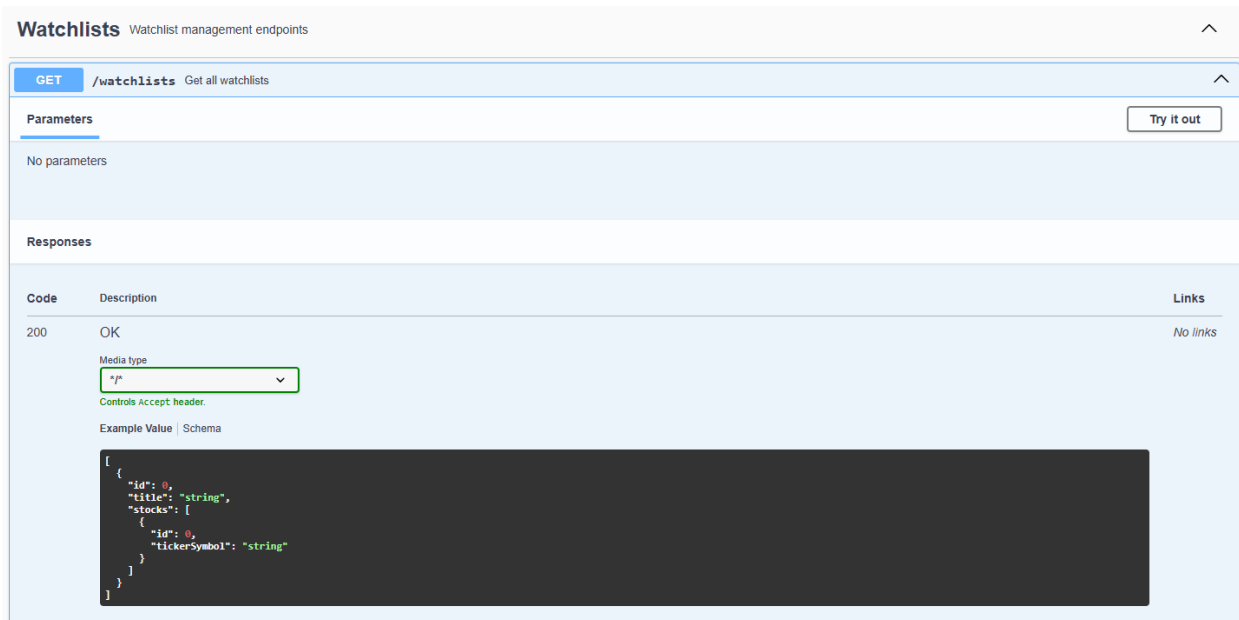


Figure 31 - Swagger get watchlist endpoint documentation

This endpoint returns the full list of watchlists upon request. In the Swagger UI, this endpoint is documented with its HTTP method, path, and expected response codes. This makes it easier to understand how the endpoint can be called and what result can be expected. Additionally, Swagger allows the endpoint to be tested interactively, which was helpful during implementation and verification of the backend.

Logging:

As the logging implementation for this project, I employed the SLF4J façade library. SLF4J stands for Simple Logging Facade for Java. It provides a standard logging interface for Java applications and is commonly used with Spring Boot. In this project, it is used for logging, in order to improve traceability. In other words, it makes it easier to understand the important events occurred during the runtime of this Spring Boot application. It aids in troubleshooting and application maintenance.

Implementation examples:

```
import org.slf4j.Logger;  
import org.slf4j.LoggerFactory;  
import org.springframework.stereotype.Service;  
  
@Service 4 usages  🧑 masa229 +1  
public class StockService {  
  
    private static final Logger log = LoggerFactory.getLogger(StockService.class); 3 usages
```

Figure 32 - logger implementation

In my StockService class, I implemented logging with the assistance of SLF4J in order to generate custom defined messages in the application log. To do this, I had to first add the required imports accordingly, as the green-underlined code above shows. Then I had to declare the logger as an object, as the orange-underlined code shows. After that, I used it with a custom defined string as a process in my `getStockEntityById` method to warn about a stock lookup failing and an error being thrown:

```
public Stock getStockEntityById(Long id) { 2 usages  🧑 masa229  
    return stockRepository.findById(id)  
        .orElseThrow(() -> {  
            log.warn("Stock lookup failed for id={}", id);  
            return new StockNotFoundException(id);  
        });  
}
```

Figure 33 - logging implementation

Exception handling:

Throughout the entirety of the backend code, basic error handling has been implemented through an exception-catching basis. Generally, common Java exceptions are used, but for the scope of this project, I also had to implement custom exceptions.

Here are the custom exceptions implemented in the backend:

- WatchlistNotFoundException – exception is thrown when a requested watchlist is nonexistent


```
package app.equityinsight.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND) 2 usages ① sheung.lam
public class WatchlistNotFoundException extends RuntimeException {
    public WatchlistNotFoundException(Long id) { super("Watchlist not found, id:" + id); }
}
```

Figure 34 - WatchlistNotFoundException implementation

- StockNotFoundException – exception is thrown when a stock is not found

```
package app.equityinsight.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND) 6 usages ① masa229
public class StockNotFoundException extends RuntimeException {
    public StockNotFoundException(Long id) { super("Stock not found:" + id); }
}
```

Figure 35 - StockNotFoundException implementation

- InvalidTickerException – exception is thrown when the business rule for a ticker symbol is invalid

```
package app.equityinsight.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.BAD_REQUEST) 6 usages ① masa229
public class InvalidTickerException extends RuntimeException {
    public InvalidTickerException(String ticker) { 4 usages ① masa229
        super("Ticker symbol '" + ticker + "' is invalid. It must contain 1 to 4 letters or numbers.");
    }
}
```

Figure 36 - InvalidTickerException implementation

- CommentNotFoundException – exception is thrown when the requested comment is not found

```
package app.equityinsight.exception;

import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.ResponseStatus;

@ResponseStatus(HttpStatus.NOT_FOUND) 3 usages @sheung.lam
public class CommentNotFoundException extends RuntimeException {
    public CommentNotFoundException(Long id) { 2 usages @sheung.lam
        super("Comment not found, id:" + id);
    }
}
```

Figure 37 - CommentNotFoundException implementation

One more thing worth mentioning is the mock data generation logic.

```
private List<PricePointDto> generatePrices(String ticker, int days) { 1 usage @masa229
    List<PricePointDto> prices = new ArrayList<>();
    Random random = new Random();
    LocalDate startDate = LocalDate.now().minusDays(daysToSubtract: days - 1);
    double currentPrice = 40 + random.nextDouble() * 210;
    for (int i = 0; i < days; i++) {
        double trend = (random.nextDouble() - 0.5) * 0.8;
        double dailyChange = trend + (random.nextDouble() - 0.5) * 4.0;
        currentPrice = Math.max(5.0, currentPrice + dailyChange);
        BigDecimal roundedPrice = BigDecimal.valueOf(currentPrice)
            .setScale(newScale: 2, RoundingMode.HALF_UP);
        prices.add(new PricePointDto(
            startDate.plusDays(i),
            roundedPrice
        ));
    }

    return prices;
}
```

Figure 38 - mock stock data implementation

For this application, backend generated data was ultimately chosen for the demonstration of a minimum viable product. However, the stock prices and their developments wouldn't be believable or resembling real world data. Therefore, an algorithm was created to ensure the generated data displays more realistically. Data sets generated with this method will:

- Have a start price between 40 and 250 (dollars)

- Have an artificial “trend” that decides whether the price gently rises or lowers to avoid too dramatic “swings”
- Never drop below 5 dollars to prevent unrealistic negative values or near-zero values
- Have the prices rounded up to two decimals

4. Frontend

The frontend was implemented with Angular and Typescript and was designed as the user interface of the application. Its purpose is to present the watchlist, selected stocks, stock charts and comments in a clear and usable way.

The frontend implementation was not started with visual design alone, but with the structural preparation of the application. Routing, environment configuration, component structure, and service classes were prepared first so that the user interface could later be connected to a functioning backend instead of being built in isolation.

A design decision from the planning phase was to use a “panel-based” structure. Instead of keeping the full user interface in a single page component, the frontend was divided into designated areas for watchlists, stock details and comments. This ensures that a single component isn’t bearing too many responsibilities. This structure also matches the results of earlier mock-ups.

The implementation of the user interface itself turned out to be more time-consuming than originally expected. While the frontend logic and service layer could be implemented relatively well, the actual UI construction in Angular was difficult due to limited practical experience in this specific area. To avoid losing too much time during the IPA, ChatGPT was used to create an initial user interface skeleton. The skeleton was then adjusted and connected to the logical layer of the frontend application. Apache ECharts was used to display stock price history as planned.

Implemented frontend areas:

- Watchlist area
 - display of watchlists
 - creation, editing, and deletion through service calls

- selection of the active watchlist
- Stock detail area
 - display of stocks in the selected watchlist
 - selection of a stock
 - chart visualisation for the selected stock
 - support for mocked chart data via backend
- Comment area
 - display of comments for a selected stock
 - creation, editing, and deletion of comments
 - temporary mock behaviour where full integration was not yet completed
- General interface features
 - panel-based layout
 - routing and environment setup
 - theme toggle support

Implemented frontend structure:

Frontend element	Purpose in the project
Components	Display UI sections and react to user input
Services	Communicate with backend endpoints
Models / DTO interfaces	Define data structures used in the frontend
Routing	Controls navigation / page structure
ECharts	Displays stock history graphically

Important frontend implementation adjustments:

Topic	Initial idea	Final implementation
User interface structure	More centralised dashboard structure	Panel-based structure with clearer separation
UI implementation	Fully manual construction	Started from AI-generated skeleton, then adapted
Chart data	Live external stock data	Backend-generated mock stock data
Frontend state	Basic local logic	Structured through services and selected panel states

Main frontend implementation steps

- Preparation of routing, environment configuration, and overall structure
- Creation of frontend services for backend communication
- Implementation of the panel-based component layout
- Construction of the UI skeleton and further adaptation
- Integration of chart visualisation with Apache ECharts
- Connection of components to services and selected backend endpoints
- Adjustment of the chart feature to work with backend-generated mock data

UI implementation:



Figure 39 - UI implementation

5. Containerisation

As explained in the database section, the database is successfully containerised and served with Podman Desktop. Please refer to section 2.3.4.2.

The backend component could not be containerised due to time constraints.

6. AI usage

Throughout the work on this IPA, AI has been used in the following ways:

- Get understanding on the IPERKA structure and recommendation on possible subchapters for my document.
- Get examples for some sections of IPERKA so that I can understand how to implement them.
- Get code examples and references for various technical topics that I had to Implement in the backend and frontend components. (examples: comparison in usage of logging services like log4j, example of mocked data structures for Apache ECharts)
- Research decision methodologies and get examples on how to apply them.

5. Control (Kontrollieren)

1. Testing

As part of the endpoint manual testing efforts, I employed two different strategies:

- Direct browser endpoint access for GET requests
- Manual Postman tests

Before I began implementing the frontend, I had to verify the correct exposure of the backend endpoints, as well as the correct implementation of them. To achieve this, Postman tests were created for each endpoint. Below are every test requests I have created, which can be run manually and effectively with the help of the Postman runner functionality:

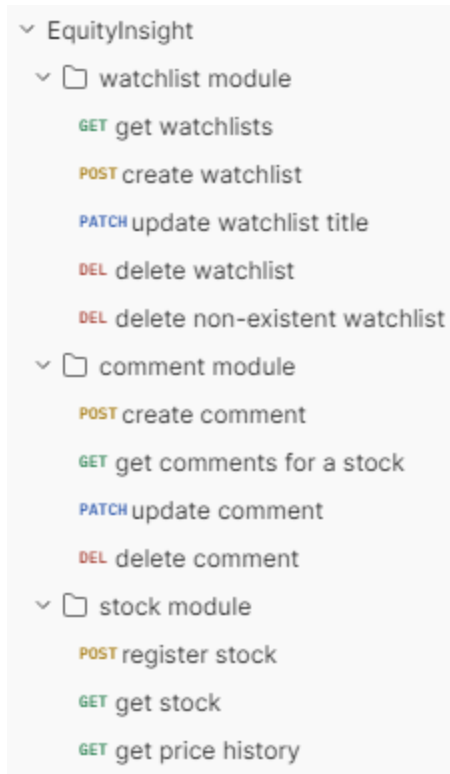


Figure 40 - Postman test list

Above are all the tests created to thoroughly test the endpoints. I will also list a few examples:

- Creating a valid watchlist:

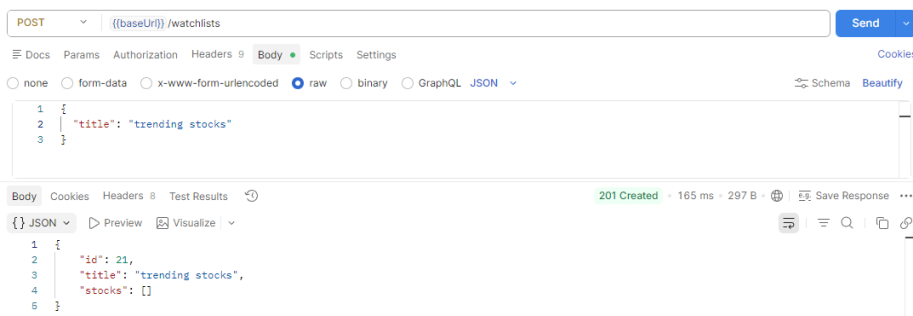


Figure 41 - valid watchlist test

The backend returns a “201 Created” status and the created watchlist as a DTO

- Creating an invalid watchlist:

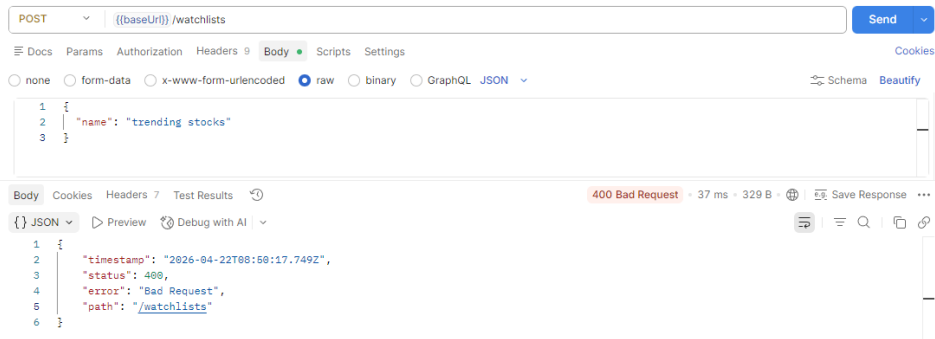


Figure 42 - invalid watchlist test

The backend returns a “400 Bad Request” status, the time stamp and the path the request was sent to

- Retrieving stock price history for a valid ticker

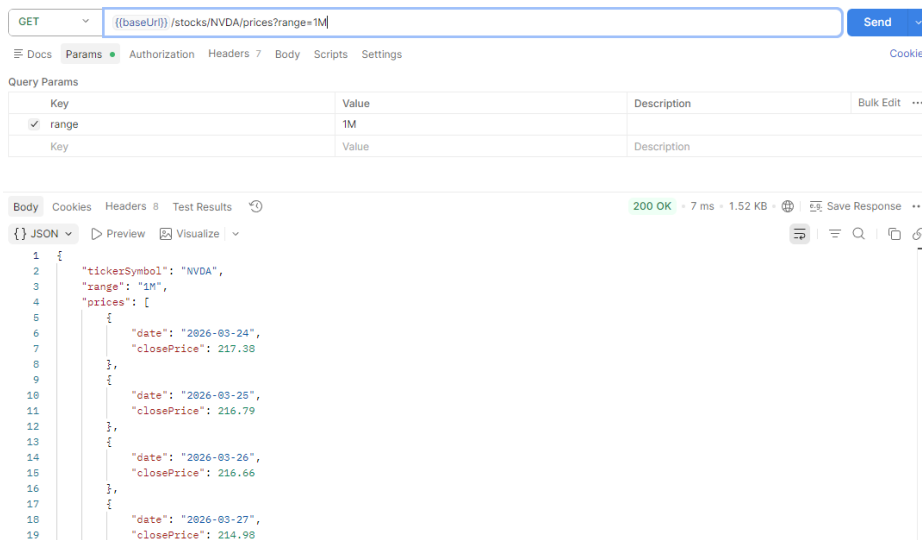


Figure 43 - valid ticker test

The backend returns a “200 OK” status, a list of “pricePointDTO” (contains date and closing price of that date), the selected time range and ticker symbol.

- Retrieving stock price history for an invalid ticker (6 characters, whereas ticker symbols can only have 4 or fewer characters):

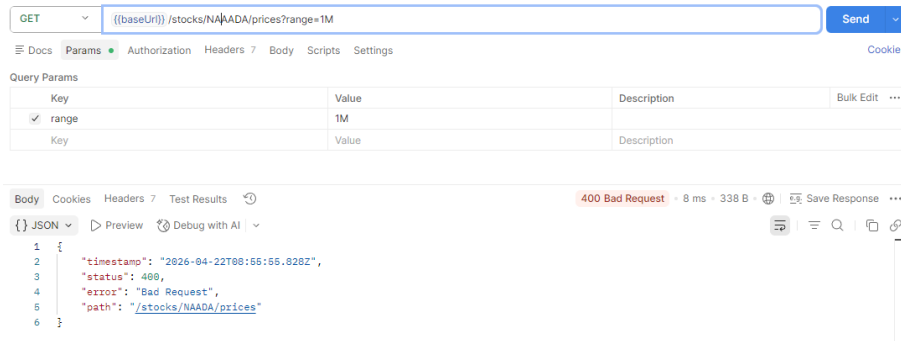


Figure 44 - invalid ticker test

The backend returns a “400 Bad Request” status, the path and the timestamp

- Creating a valid comment

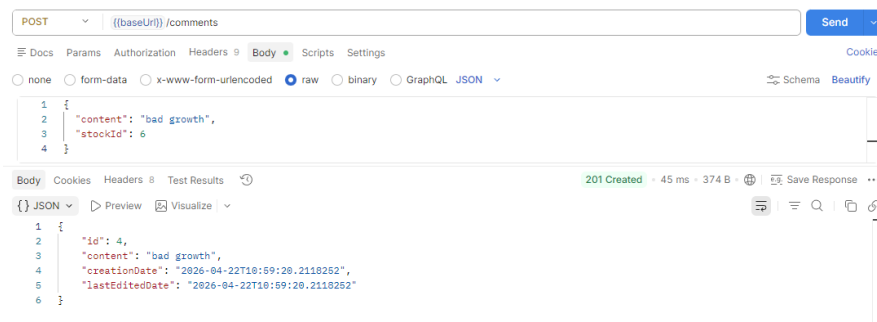


Figure 45 - valid comment test

The backend returns a “201 Created” status and the comment as a DTO

- Updating a comment

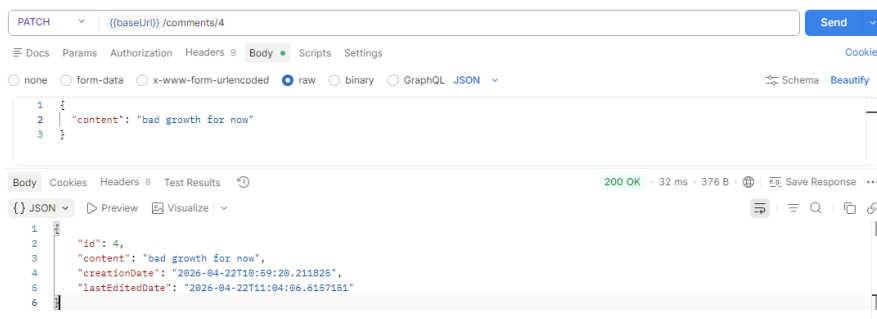


Figure 46 - invalid comment test

The backend returns a “200 OK” status and the updated comment as a DTO, where its “lastEditedDate” field is adjusted accordingly

- Deleting a comment

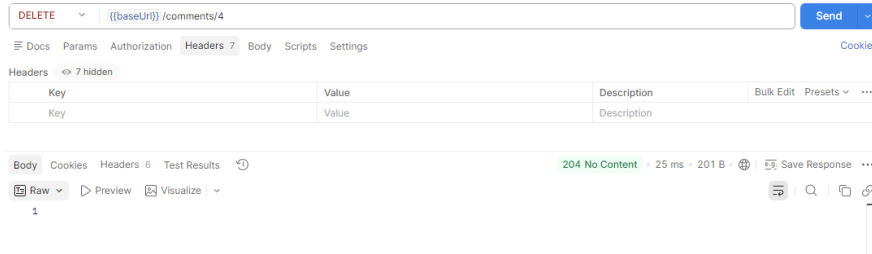


Figure 47 - delete comment test

The backend returns a “204 No Content”

As for the originally planned unit tests and end-to-end tests, their inclusion in the testing phase of this project had to be abandoned due to timing constraints.

6. Evaluate (Auswerten)

In this section, I will be comparing the final result of this project with the originally defined requirements. The purpose is to clearly display which requirements are fulfilled, which are partially fulfilled, and which are not fulfilled.

Requirement	Status	Implementation	Comment
Create watchlists	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented
View watchlists	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented
Update watchlists	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented
Delete watchlists	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented

Add stocks to watchlists	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented
Search stocks by ticker symbol	fulfilled	Endpoint created and accessed from the frontend	Verification of real ticker symbols lacks due to lack of external data source
Search stocks by company name	not fulfilled	Not implemented due to lack of external data source	
Display historical stock price in charts	fulfilled	Mock stock price displayed in the frontend using Apache ECharts	Core functionality implemented
Support different time ranges for stock price display	fulfilled	Time range toggle integrated into Apache ECharts	Core functionality implemented
Create comments for a stock	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented
Edit comments	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented
Delete comments	fulfilled	Endpoint created and accessed from the frontend	Core functionality implemented
Include timestamps for comments	fulfilled	Timestamp for both the creation and last edit	Core functionality implemented
Switch between light mode and dark mode at runtime	not fulfilled	Feature not implemented due to time restraints	
Persist locally managed user data	fulfilled	All user-related data stored in local database	Core functionality implemented
Frontend implementation with Angular and TypeScript	fulfilled	Frontend application is built according to the official Angular documentation	
Backend implementation with Java and Spring Boot	fulfilled	Backend application is built according to the official Spring Boot documentation	
Database implemented as PostgreSQL	fulfilled	The database is created with a Podman command	

		for a stable PostgreSQL version	
Chart visualisation implemented with Apache ECharts	fulfilled	Apache ECharts is integrated into the frontend application to display mock stock prices	
Database run with Podman	fulfilled	Database runs in a Podman container	
Modular architecture with clear separation between frontend, backend and database	fulfilled	The frontend, backend and database layers are clearly separated as documented	
Backend validates incoming requests	fulfilled	Requests are validated by the backend and custom error are thrown for invalid requests	Core functionality implemented
Application remains understandable and maintainable	fulfilled	The application structure is designed and implemented with maintainability in focus	
Charts and UI interactions run within a reasonable time frame	fulfilled	Chart and UI interactions run smoothly in manual tests	
Containerisation of the backend component	not fulfilled	The backend application has not been containerized due to time constraints	
Application is tested thoroughly	partially fulfilled	Unit testing and E2E testing had to be abandoned due to time constraints, manual testing and API integration testing has been done successfully	

4. List of illustrations

Figure 1 - time plan.....	12
Figure 2 - ERD V1	32
Figure 3 - ERD V2.....	32
Figure 4 - ERD V3.....	33
Figure 5 - frontend structure V1	34
Figure 6 - frontend structure V2	34
Figure 7 - backend structure V1.....	35
Figure 8 - backend structure V2.....	35
Figure 9 - backend structure V3.....	36
Figure 10 - backend structure V4.....	36
Figure 11 - dashboard mock-up V1	38
Figure 12 - stock search mock-up V1	38
Figure 13 - dashboard mock-up V2	39
Figure 14 - stock view mock-up V2	40
Figure 15 - watchlist view mock-up V2	40
Figure 16 - add to watchlist dialog mock-up V2	41
Figure 17 - dashboard mock-up V3	42
Figure 18 - stock view mock-up V3	42
Figure 19 - watchlist view mock-up V3	43
Figure 20 - add to watchlist dialog mock-up V3	43
Figure 21 - feedback request for V2	44
Figure 22 - feedback request for V3	45
Figure 23 - Swagger endpoint list.....	47
Figure 24 - backend structure V1.....	53
Figure 25 - backend structure V2.....	54
Figure 26 - backend structure V3.....	54
Figure 27 - backend structure V4.....	54
Figure 28 - application structure	56
Figure 29 - Swagger endpoint list.....	61
Figure 30 - logger implementation	63
Figure 31 - logging implementation	63
Figure 32 - WatchlistNotFoundException implementation.....	64
Figure 33 - StockNotFoundException implementation	64
Figure 34 - InvalidTickerException implementation.....	64
Figure 35 - CommentNotFoundException implementation	65
Figure 36 - mock stock data implementation.....	65
Figure 37 - UI implementation	68
Figure 38 - Postman test list.....	70
Figure 39 - valid watchlist test.....	70
Figure 40 - invalid watchlist test.....	71

Figure 41 - valid ticker test	71
Figure 42 - invalid ticker test	72
Figure 43 - valid comment test	72
Figure 44 - invalid comment test	72
Figure 45 - delete comment test.....	73

5. List of sources

Source	Link
Decision tables and decision trees	https://medium.com/analysts-corner/its-only-logical-decision-tables-and-decision-trees-12a8b52243ea
slf4j user manual	https://www.slf4j.org/manual.html
Knockout criteria methodology	https://www.projectmanagement.com/process/processmain.cfm?ID=9242#
Rest endpoint documentation	https://www.baeldung.com/spring-rest-openapi-documentation
Apache ECharts	https://echarts.apache.org/en/index.html
Stooq	https://stooq.com/
Angular documentation	What is Angular? • Angular
Spring Boot Framework documentation	Annotated Controllers :: Spring Framework
Spring Bean validation	Java Bean Validation :: Spring Framework
Swagger documentation	OpenAPI 3 Library for spring-boot
Draw.io	Untitled Diagram - draw.io
Figma	Login Figma

6. Glossary

Term	Definition
API	Interface for communication between software components
CRUD	Create, Read, Update, Delete

DTO	Object used to transfer data between layers
E2E tests	Test of a complete user workflow from start to finish
Endpoint	A specific backend URL for an API request
Entity	Backend object mapped to a database table
ERD	Diagram showing entities and their relationships
IPERKA	Project method with the phases Inform, Plan, Decide, Realise, Control, Evaluate
JPA	Java standard for mapping objects to database tablese
Logging	Recording application events and errors for debugging
MVC	Architecture pattern separating model, view, and controller
Podman	Tool for running software in containers
PostgreSQL	Relational database used for data storage
Repository	Backend component for database access
REST	Architectural style for web APIs using HTTP methods
SLF4J	Java logging interface used in the backend
Spring Boot	Java framework used for the backend application
Stock	A type of security that gives the older a share of company ownership
Swagger	Tool for documenting and testing REST endpoints
Swing	Significant price change of a stock
Ticker Symbol	Short code used to identify a stock
Trend	Tendency of a stock to move in a particular direction
Watchlist	User-defined list of selected stocks

7. Attachment

- Code base